N046 AI 서비스 인 프로덕션_해설서

Chapter01

강의정리 챕터 01. Google Cloud에 처음 배포하기

1. 오늘 강의의 목적

지금까지 배운 AI 에이전트를 내 컴퓨터에서만 실행하는 수준에서 벗어나, 다른 사람도 인터넷으로 접속해 사용할 수 있는 서비스로 만드는 것이 이번 강의의 목적입니다.

예를 들어, 사용자가 성격과 말투를 설정한 AI 친구가 다음 기능을 제공한다고 생각해 보겠습니다.

- 사용자와 대화하고 이전 대화를 기억합니다.
- 날씨나 공휴일 정보를 찾아 답합니다.
- 약속을 저장하고 정해진 시점에 먼저 말을 겁니다.

이러한 기능을 하나의 서비스로 묶고 Google Cloud의 Cloud Run에 배포해 공개 주소를 만드는 것이 오늘의 핵심입니다.

2. 중학생도 이해할 수 있는 설명

① 로컬 실행과 클라우드 배포의 차이



로컬 실행: 내 방에서만 운영하는 가게

내 컴퓨터에서 프로그램을 실행해야 서비스를 사용할 수 있습니다. 컴퓨터를 끄면 서비스도 멈추고, 다른 사람에게 접속 주소를 알려 주더라도 외부에서 접근하기 어려울 수 있습니다.



클라우드 배포: 인터넷에 문을 연 가게

프로그램을 클라우드 서버에 올리면 다른 사람이 인터넷 주소로 접속할 수 있습니다. 내 컴퓨터가 꺼져 있어도 클라우드 환경에서 서비스를 운영할 수 있습니다.

쉽게 말해, 배포는 내가 만든 프로그램을 다른 사람도 사용할 수 있도록 인터넷에 올리는 과정입니다.

② 배포 과정에서 Google Cloud는 무슨 일을 할까요?

내 컴퓨터

프로그램을 개발하고 먼저 실행해 보기

Google Cloud · Cloud Run

프로그램을 실행할 클라우드 환경 제공

공개 서비스 주소

동료와 사용자가 인터넷으로 접속

Google Cloud는 서버와 실행 환경을 제공하는 플랫폼이고, Cloud Run은 그 안에서 웹 서비스를 실행할 수 있도록 해 주는 서비스입니다.

이번 실습에서는 Python 웹 프레임워크인 FastAPI로 만든 에이전트를 Cloud Run에 올립니다.

3. 핵심 개념 4가지

개념	쉬운 설명	이번 강의에서의 역할
FastAPI	웹 서비스를 만드는 도구	AI 에이전트의 기능을 웹으로 제공
Google Cloud	인터넷상의 컴퓨터와 여러 서비스를 빌려 쓰는 플랫폼	서비스 운영 기반
Cloud Run	웹 프로그램을 실행해 주는 클라우드 서비스	공개 주소로 서비스 제공
Secret Manager	중요한 비밀 정보를 보관하는 금고	LLM API 키 안전하게 보관

특히 중요한 것: LLM API 키

AI 에이전트가 OpenAI나 Google 등의 언어 모델을 이용하려면 API 키가 필요할 수 있습니다. 이 키는 AI 서비스를 호출할 권한과 사용량 비용을 연결하는 중요한 정보입니다.

API 키를 소스 코드에 직접 적거나 GitHub 같은 공개 저장소에 올리면 다른 사람이 키를 악용할 수 있습니다.

따라서 이번 강의에서는 API 키를 코드와 분리해 Secret Manager라는 안전한 보관 장소에 저장하는 방법을 배웁니다.

4. 실제 배포 과정에서 일어나는 일

배포 버튼을 누른다고 해서 프로그램이 저절로 인터넷에 나타나는 것은 아닙니다. 일반적으로 다음과 같은 과정이 진행됩니다.

1. 프로그램 준비: FastAPI 코드와 필요한 라이브러리, 실행 설정을 준비합니다.
2. 실행 환경 구성: Cloud Run에서 프로그램을 실행할 수 있도록 컨테이너 이미지 등을 준비합니다.
3. 클라우드 배포: 준비한 프로그램을 Google Cloud에 올리고 실행합니다.

4. 접속 주소 생성: 서비스에 접근할 수 있는 URL을 제공합니다.
5. 비밀 정보 연결: Secret Manager에 저장한 API 키를 서비스가 안전하게 사용할 수 있도록 권한과 설정을 구성합니다.
6. 서비스 확인: 공개 주소로 접속해 에이전트가 정상적으로 동작하는지 확인합니다.

실제 순서와 세부 작업은 배포 방식 및 프로젝트 설정에 따라 달라질 수 있습니다.

5. 왜 AI 서비스 개발자가 알아야 할까요?

AI 에이전트는 단순히 답변을 잘 만드는 것만으로는 실제 서비스가 되지 않습니다. 사용자가 지속적으로 이용할 수 있도록 운영 환경까지 갖춰야 합니다.

이번 강의에서 알아야 할 핵심은 다음 세 가지입니다.

- 접근성: 다른 사람이 접속할 수 있는 주소가 있어야 합니다.
- 보안: API 키와 같은 민감한 정보를 안전하게 관리해야 합니다.
- 운영 효율: 응답 속도와 API 호출 비용을 관리해야 합니다.

마지막 항목은 다음 실습에서 더 자세히 다룹니다. 에이전트가 메시지 하나에 LLM을 여러 번 호출하면 응답이 늦어지고 비용도 늘어날 수 있기 때문입니다.

6. 나의 AI 사업 프로젝트에 적용하기

현재 구상하고 계신 프로젝트를 기준으로 보면, 이번 강의는 특히 My-Story-Maker(MSM) 에 적용하기 좋습니다.

프로젝트	클라우드 배포 후 가능한 일
MSM	사용자가 글을 입력하고 AI가 문체를 분석하거나 이야기를 생성하는 서비스 운영
AI Story Partner(ASP)	홈페이지에서 AI 컨설팅 서비스와 결과물을 제공
뉴스레터 에이전트	정해진 업무 흐름에 따라 정보를 수집하고 결과를 생성하는 서비스 운영

MSM을 예로 들면, 처음에는 내 컴퓨터에서 글을 입력하고 AI가 분석하는 기능을 시험합니다. 이후 클라우드에 배포하면 다른 사용자에게 주소를 보내 직접 테스트를 부탁할 수 있습니다.

다만 공개 주소가 생겼다고 해서 회원 관리, 데이터베이스, 개인정보 보호, 비용 통제까지 자동으로 완성되는 것은 아닙니다. 이러한 요소는 별도로 설계해야 합니다.

7. 주관식 퀴즈 모범 답안

질문: Vercel에서 배포 버튼을 누른 뒤 공개 주소가 생기기까지 어떤 일이 일어났을까요?

다음과 같이 작성하시면 됩니다.

> Vercel은 먼저 내가 만든 프로그램의 코드와 설정 파일을 확인하고, 서비스를 실행하는 데 필요한 환경을 준비했을 것입니다. 필요한 파일과 라이브러리를 구성하고 프로그램을 빌드한 뒤, Vercel의 서버 환경에 배포해 실행할 수 있도록 했을 것입니다. 이후 외부에서 접속할 수 있는 URL을 연결해 주었을 것입니다. 그래서 내 컴퓨터에서 직접 프로그램을 실행하지 않아도 다른 사람이 인터넷으로 서비스에 접속할 수 있게 된 것 같습니다.
>
> 다만 배포가 완료되었다고 해서 모든 기능이 자동으로 작동하는 것은 아닙니다. API 키, 환경변수, 데이터베이스 연결 등이 올바르게 설정되어 있어야 실제 서비스가 정상적으로 동작할 것입니다.

참고: 이는 일반적인 Vercel 배포 과정에 대한 추론입니다. 실제로 어떤 빌드와 실행 과정이 수행되었는지는 프로젝트의 프레임워크와 배포 설정에 따라 달라집니다.

8. 반드시 기억할 내용

- 배포: 내 컴퓨터에서 만든 프로그램을 다른 사람이 이용할 수 있는 환경에 올리는 것
- Cloud Run: Google Cloud에서 웹 서비스를 실행하는 서비스
- Secret Manager: API 키와 같은 비밀 정보를 안전하게 보관하는 서비스
- 운영 준비: 공개 주소뿐 아니라 보안, 응답 속도, 비용까지 고려해야 실제 서비스가 됨

한 문장 정리

AI 에이전트 개발은 프로그램을 만드는 데서 끝나지 않으며, 클라우드에 배포하고 보안과 운영 비용까지 관리해야 실제 사용자가 이용할 수 있는 서비스가 됩니다.

Chapter02

강의정리 챕터 02. 왜 Cloud Run으로 배포할까?

1. 오늘 강의의 목적

이번 강의의 핵심은 Vercel과 Cloud Run의 차이를 이해하고, 왜 AI 에이전트 서비스에 컨테이너 기반 배포가 유리한지 알아보는 것입니다.

Vercel로 웹사이트를 배포할 때는 저장소를 연결하고 버튼을 누르면 공개 주소가 생겼습니다. 그렇다면 AI 에이전트도 똑같이 배포하면 될까요?

간단한 서비스라면 가능하지만, AI 에이전트는 여러 번 AI 모델을 호출하거나 문서를 읽고 분석하는 등 오래 걸리는 작업을 수행할 수 있습니다. 또한 특정 Python 라이브러리나 운영 체제 프로그램이 필요할 수도 있습니다.

이런 상황에서는 프로그램의 실행 환경을 직접 구성할 수 있는 Cloud Run의 컨테이너 방식이 유용합니다.

2. 중학생도 이해할 수 있는 설명

① Vercel과 Cloud Run은 무엇이 다를까요?

식당을 운영한다고 생각해 보겠습니다.



Vercel: 준비된 주방을 이용하는 식당

플랫폼에서 정해 놓은 주방과 조리 도구를 이용합니다. 메뉴에 해당하는 코드를 올리면 플랫폼이 상당 부분을 알아서 준비합니다. 편리하지만 사용할 수 있는 도구와 조리 환경에 제약이 있습니다.



Cloud Run: 필요한 도구를 직접 챙기는 식당

코드뿐 아니라 실행 환경과 필요한 도구까지 컨테이너라는 상자에 담아 올립니다. 원하는 환경을 더 세밀하게 구성할 수 있지만, 필요한 도구를 빠짐없이 준비하고 관리해야 합니다.

두 방식 모두 서버를 직접 관리하는 부담을 줄여 주는 서버리스(serverless) 방식입니다. 가장 큰 차이는 서버가 있느냐 없느냐가 아니라, 프로그램 실행 환경을 누가 어느 범위까지 준비하느냐입니다.

② 컨테이너에는 무엇이 들어갈까요?

컨테이너 이미지

프로그램을 실행하기 위해 필요한 구성 요소를 담은 패키지

코드

프론트엔드와 백엔드 프로그램

런타임(Runtime)

Python처럼 코드를 실행하는 환경

의존성(Dependencies)

FastAPI 등 코드가 사용하는 라이브러리

시스템 패키지

문서 변환 도구 등 운영체제에 설치해야 하는 프로그램

예를 들어 AI 에이전트가 PDF 문서를 읽고 요약해야 하는데 특정 문서 처리 프로그램이 필요하다면, 해당 프로그램을 컨테이너 이미지에 포함할 수 있습니다.

다만 API 키 같은 비밀 정보는 일반적으로 이미지에 직접 넣지 않고 Secret Manager 등의 별도 보안 저장소에서 관리합니다.

3. 핵심 개념 5가지

개념	쉬운 설명	알아야 하는 이유
서버리스	서버 운영을 플랫폼이 대신 관리하는 방식	서버 관리 부담을 줄임
컨테이너 이미지	실행에 필요한 구성 요소를 묶은 패키지	환경 차이로 생기는 오류를 줄임
오토스케일링	요청량에 따라 실행 인스턴스 수를 조절	이용자 증가에 대응
스케일 투 제로	요청이 없을 때 인스턴스를 0개까지 줄임	유휴 인스턴스 비용 절감 가능
요청 타임아웃	요청 하나를 처리할 수 있는 최대 시간	오래 걸리는 AI 작업이 중간에 끊기는 것을 방지

특히 알아둘 개념: 인스턴스와 동시성

- 인스턴스(Instance): 프로그램을 실행하는 실제 실행 단위입니다. 식당으로 비유하면 일하는 주방 한 팀과 비슷합니다.
- 동시성(Concurrency): 인스턴스 하나가 동시에 처리하도록 허용하는 요청 수입니다. 주방 한 팀이 동시에 처리할 수 있는 주문 수라고 생각하시면 됩니다.

요청이 많아지면 플랫폼은 인스턴스를 추가할 수 있습니다. 반대로 요청이 줄어들면 인스턴스 수를 줄일 수 있습니다.

다만 스케일 투 제로가 가능하더라도 실제 비용은 CPU 할당 방식, 최소 인스턴스 설정, 네트워크 사용량 등 여러 조건에 따라 달라집니다.

4. 직접 비교하기: Vercel vs. Cloud Run

비교 항목	Vercel	Cloud Run
기본 접근 방식	플랫폼이 빌드·실행 환경을 상당 부분 관리	컨테이너 이미지에 실행 환경을 구성
실행 환경 선택	플랫폼이 지원하는 범위 내에서 선택	이미지에 포함할 환경을 더 세밀하게 제어
시스템 패키지	플랫폼의 실행 조건과 지원 범위에 영향받음	필요한 패키지를 이미지에 포함 가능
실행 시간	배포 유형과 요금제별 제한 적용	요청 타임아웃을 설정할 수 있음
관리 편의성	간편한 배포와 자동화에 강점	컨테이너 구성과 운영 설정에 대한 이해 필요
적합한 활용 예	프론트엔드, 지원되는 웹 함수	사용자 정의 실행 환경이 필요한 백엔드·AI 에이전트

강의 자료에서는 2026년 10월 기준 Vercel 무료 플랜의 함수 요청 타임아웃을 최대 5분, Cloud Run의 요청 타임아웃을 기본 5분에서 최대 60분으로 설명합니다. 다만 실제 제한은 제품 설정과 실행 유형에 따라 다를 수 있으므로, 배포 시점의 공식 설정을 확인해야 합니다.

중요한 것은 무조건 Cloud Run이 우월하다는 뜻이 아니라, 내 서비스가 요구하는 실행 시간과 환경에 맞춰 배포 방식을 선택하는 것입니다.

5. 왜 AI 에이전트에 중요할까요?

AI 에이전트가 사용자의 요청을 처리하는 과정은 일반적인 웹페이지를 보여 주는 것보다 복잡할 수 있습니다.

예를 들어 사용자가 “최근 뉴스 20개를 조사하고, 서로 비교한 뒤 보고서를 작성해 줘”라고 요청했다고 가정해 보겠습니다.

사용자 요청

뉴스 20개를 조사해 보고서 작성

자료 수집 및 분석

검색 · 문서 읽기 · LLM 호출

결과 생성

비교 · 검증 · 보고서 작성

사용자에게 결과 전달

이 과정에서는 외부 API 응답이 늦어지거나, 여러 차례 모델을 호출하거나, 문서를 처리하는 데 시간이 오래 걸릴 수 있습니다.

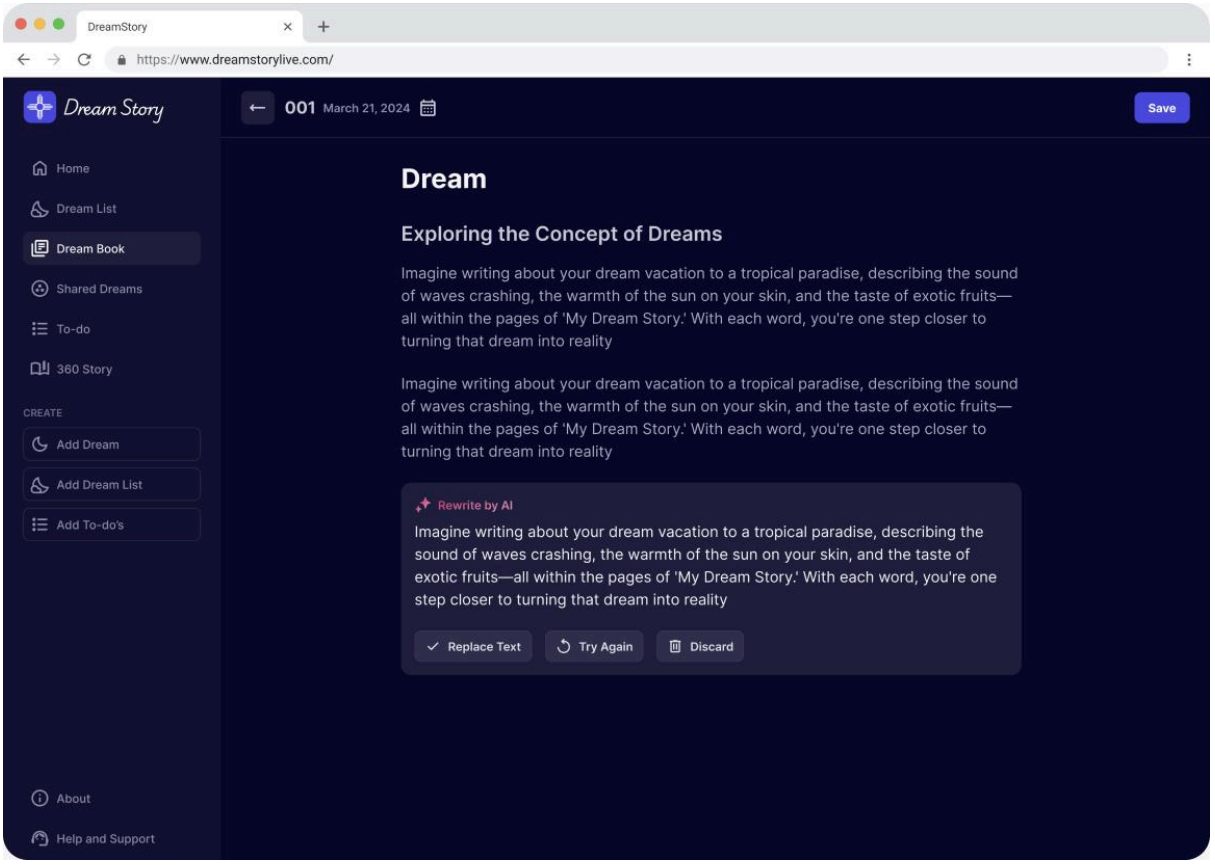
따라서 다음 세 가지를 함께 고려해야 합니다.

1. 실행 시간: 작업이 요청 타임아웃 안에 끝나는가?
2. 실행 환경: 필요한 라이브러리와 시스템 패키지가 준비되어 있는가?
3. 운영 비용: 모델 호출 비용과 클라우드 실행 비용을 감당할 수 있는가?

참고로 타임아웃을 길게 설정한다고 해서 작업이 반드시 성공하는 것은 아닙니다. 장시간 작업은 작업 상태 저장, 재시도, 백그라운드 처리 등을 별도로 설계해야 할 수도 있습니다.

6. 나의 프로젝트에 적용하기

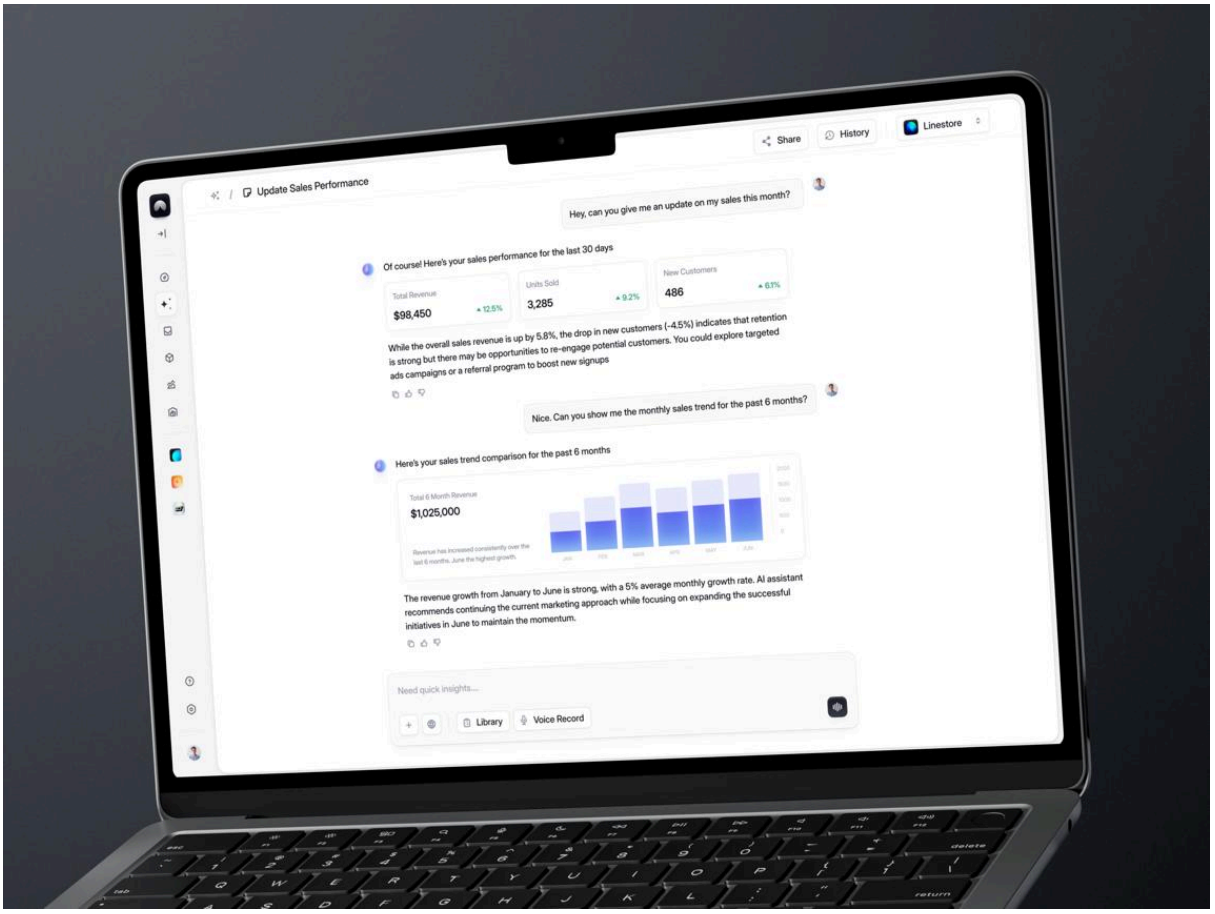
현재 진행 중인 프로젝트 중에서는 My-Story-Maker(MSM) 에 가장 직접적으로 연결할 수 있습니다.



MSM — My-Story-Maker

사용자가 글을 입력하면 AI가 문체를 분석하고, 원문의 특징을 유지한 채 다듬은 글과 웹소설을 생성하는 서비스입니다.

- 초기에는 FastAPI와 필요한 라이브러리를 컨테이너에 넣어 Cloud Run에 배포합니다.
- 문체 분석처럼 짧은 작업부터 공개 환경에서 시험합니다.
- 긴 원고 변환은 타임아웃과 비용을 측정한 뒤 필요하면 비동기 작업 구조를 검토합니다.



ASP — AI Story Partner

AI 컨설팅 문의를 받고 사업 진단이나 콘텐츠 전략을 제공하는 서비스입니다.

- 홈페이지와 AI 처리 서버를 분리할지 함께 배포할지 선택할 수 있습니다.
- 긴 분석 보고서 생성은 실행 시간뿐 아니라 모델 호출 비용과 결과 저장 방식도 고려해야 합니다.

이번 강의의 실습 대상인 AI 친구 에이전트는 FastAPI가 화면과 서버 기능을 함께 제공하는 구조이므로 Cloud Run 서비스 하나로 시작하기 좋습니다. 추후 화면이 복잡해지면 프론트엔드를 Vercel에, 백엔드를 Cloud Run에 분리하는 방식도 검토할 수 있습니다.

7. 이번 강의에서 기억할 내용

- Vercel은 배포 환경을 상당 부분 대신 관리해 주므로 시작하기 편리합니다.
- Cloud Run은 컨테이너에 실행 환경을 담아 올리므로 환경을 더 세밀하게 제어할 수 있습니다.
- 두 플랫폼 모두 서버리스 방식이며, 서버 관리 부담을 줄이는 것이 목적입니다.

- AI 에이전트는 긴 실행 시간과 추가 라이브러리 때문에 배포 환경을 신중하게 선택해야 합니다.
- 컨테이너를 직접 관리하면 자유도가 커지는 대신 구성 오류와 운영 책임도 늘어납니다.

8. 한 문장 정리

Vercel은 플랫폼이 준비한 환경을 편리하게 활용하는 방식이고, Cloud Run은 실행 환경을 컨테이너로 직접 구성해 AI 에이전트에 필요한 자유도와 제어력을 확보하는 방식입니다.

Chapter03

강의정리 챕터 03. Google Cloud가 하는 일

1. 오늘 강의의 목적

이번 강의의 목적은 Google Cloud에 AI 에이전트를 배포하기 전에 필요한 계정, 프로젝트, 결제, 권한, 개발 도구를 준비하는 것입니다.

앞선 강의에서 Cloud Run이 AI 에이전트를 실행하는 곳이라는 점을 배웠습니다. 이번에는 그 서비스를 사용하려면 Google Cloud에서 무엇을 준비해야 하는지 알아봅니다.

오늘의 핵심 작업은 다음과 같습니다.

1. Google Cloud 무료 체험과 결제 계정을 확인합니다.
2. 실습용 프로젝트를 만듭니다.
3. `gcloud CLI`를 설치하고 로그인합니다.
4. 기본 프로젝트와 리전을 설정합니다.
5. Cloud Run 등 필요한 API를 활성화합니다.

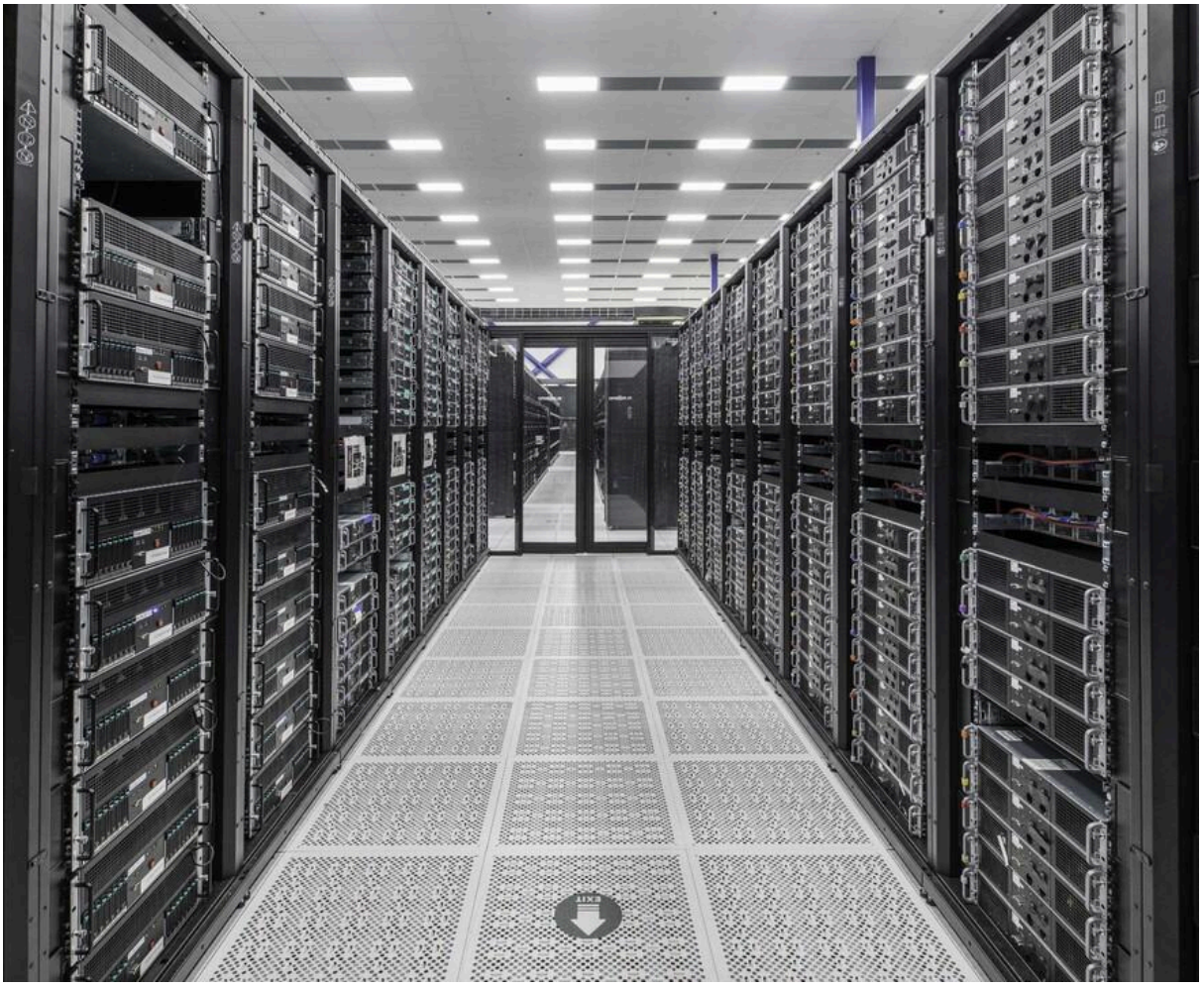
아직 AI 에이전트를 실제로 배포하는 단계는 아닙니다. 배포할 수 있도록 작업 공간과 도구를 준비하는 단계입니다.

2. 중학생도 이해할 수 있는 설명

① Google Cloud는 무엇인가요?

Google Cloud는 Google이 운영하는 데이터센터의 컴퓨터, 저장 공간, 네트워크 등을 필요한 만큼 빌려 쓰는 서비스입니다.

내가 직접 고성능 컴퓨터를 구매해 서버실을 만들 필요 없이, Google이 제공하는 컴퓨터를 빌려 AI 서비스를 운영할 수 있습니다.



직접 서버를 운영하는 방식

컴퓨터를 구매하고, 설치하고, 고장과 네트워크를 직접 관리해야 합니다.



Google Cloud를 이용하는 방식

필요한 컴퓨팅 자원을 빌려 쓰고, 사용한 서비스와 자원에 따라 비용을 지불합니다.

AWS(Amazon Web Services), Microsoft Azure도 비슷한 종류의 클라우드 플랫폼입니다.

단, 클라우드라고 해서 모든 서비스가 무료이거나 사용하지 않을 때 반드시 비용이 0원이 되는 것은 아닙니다. 저장 공간, 네트워크, 최소 인스턴스 설정 등에는 별도 비용이 발생할 수 있습니다.

② Google Cloud의 구조 이해하기

이번 강의에서 가장 중요한 것은 Google Cloud의 계층 구조입니다.

Google 계정

누가 로그인하는가?

결제 계정

어디에서 비용을 지불하는가?

프로젝트

서비스와 설정을 실제로 관리하는 작업 공간

Cloud Run

서비스 실행

Artifact Registry

이미지 보관

Secret Manager

비밀 정보 보관

Cloud Logging

실행 기록 확인

쉽게 비유하면 다음과 같습니다.

- Google 계정은 건물에 들어오는 사람의 신분증입니다.
- 결제 계정은 건물에서 사용한 비용을 지불하는 수단입니다.
- 프로젝트는 여러 작업을 수행하는 사무실입니다.
- Cloud Run과 Secret Manager 등은 사무실 안에서 사용하는 도구입니다.

여기서 주의할 점이 하나 있습니다. 결제 계정과 프로젝트는 같은 것이 아닙니다. 결제 계정은 비용을 관리하고, 프로젝트는 서비스와 리소스를 관리합니다. 하나의 결제 계정에 여러 프로젝트를 연결할 수도 있습니다.

3. Google Cloud에서 사용하는 주요 서비스 6가지

서비스	하는 일	쉬운 비유
Cloud Run	컨테이너를 실행하고 서비스 주소 제공	가게 운영
Cloud Build	코드를 컨테이너 이미지로 빌드	제품 조립
Artifact Registry	컨테이너 이미지 저장	완제품 창고
Secret Manager	API 키 등 비밀 정보 보관	잠금 금고
IAM	사용자와 서비스의 권한 관리	출입 권한 관리
Cloud Logging	서비스의 실행 기록과 오류 확인	업무 일지

이 서비스들은 서로 연결되어 작동합니다.

예를 들어 AI 에이전트를 배포할 때는 Cloud Build가 컨테이너 이미지를 만들고, Artifact Registry에 저장한 뒤, Cloud Run이 해당 이미지를 실행하는 흐름을 사용할 수 있습니다.

API 키는 Secret Manager에 보관하고, IAM을 통해 어떤 서비스가 해당 키를 읽을 수 있는지 관리합니다. 문제가 생기면 Cloud Logging에서 실행 기록과 오류를 확인합니다.

4. 프로젝트 이름, 프로젝트 ID, 프로젝트 번호의 차이

이 부분은 실제 실습에서 오류가 발생하기 쉬우므로 반드시 구분해야 합니다.

구분	예시	특징
프로젝트 이름	`friend-agent-course`	콘솔에 표시되는 이름이며 변경 가능
프로젝트 ID	`friend-agent-course-471203`	고유 식별자이며 생성 후 변경 불가
프로젝트 번호	`123456789012`	Google Cloud가 자동으로 부여하는 숫자

프로젝트 이름은 사람이 알아보기 쉽게 붙이는 이름입니다. 프로젝트 ID는 명령어에서 프로젝트를 지정할 때 사용하는 식별자입니다. 프로젝트 번호는 일부 권한 설정이나 서비스 구성에서 사용합니다.

예를 들어 다음 명령어를 실행한다고 가정해 보겠습니다.

```
gcloud config set project friend-agent-course-471203
```

이 명령은 현재 `gcloud`가 작업할 기본 프로젝트를 지정합니다.

프로젝트 이름만 보고 명령어에 입력하면 오류가 날 수 있습니다. 콘솔 대시보드에서 프로젝트 ID를 확인한 뒤 정확하게 입력하는 습관이 중요합니다.

5. 리전과 API는 왜 설정해야 할까요?

① 리전(Region)

리전은 클라우드 서비스가 실제로 실행되는 지리적 위치입니다.

이번 실습에서는 미국 중부 지역인 `us-central1`을 사용합니다.

```
gcloud config set run/region us-central1
```

이렇게 설정하면 Cloud Run 명령을 실행할 때 기본 리전으로 활용할 수 있습니다.

리전은 단순히 명령어를 짧게 쓰기 위한 설정만은 아닙니다. 서비스의 지연 시간, 가격, 데이터 위치, 사용 가능한 기능에도 영향을 줄 수 있습니다. 따라서 실제 고객용 서비스에서는 사용자 위치와 데이터 규정 등을 고려해야 합니다.

② API 활성화

Google Cloud 프로젝트에서는 서비스를 사용하기 전에 해당 API를 활성화해야 하는 경우가 많습니다.

이번 실습에서는 다음 네 가지 API를 활성화합니다.

```
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  secretmanager.googleapis.com
```

각 API가 활성화되어야 해당 서비스의 기능을 프로젝트에서 사용할 수 있습니다.

단, API를 활성화했다고 해서 모든 기능이 자동으로 허용되는 것은 아닙니다. 결제 설정과 IAM 권한도 올바르게 갖춰져 있어야 합니다.

6. 실제 실습 순서

1단계. 무료 체험 및 결제 확인

Google Cloud 무료 체험에서 자격 조건을 확인하고 가입합니다. 콘솔의 결제 메뉴에서 크레딧과 남은 기간을 확인합니다.

2단계. 실습용 프로젝트 생성

프로젝트 이름을 정하고 생성 후 프로젝트 ID를 기록합니다. 이후 모든 명령이 이 프로젝트를 대상으로 하는지 확인합니다.

3단계. gcloud CLI 설치

`gcloud CLI`는 터미널에서 Google Cloud를 제어하는 도구입니다. Windows 또는 macOS에 맞는 방법으로 설치하고 다음 명령으로 확인합니다.

```
gcloud --version
```

4단계. 로그인 및 프로젝트 설정

```
gcloud init
gcloud config set project <프로젝트_ID>
```

```
gcloud config set run/region us-central1
```

브라우저에서 직접 로그인하고 올바른 프로젝트를 선택합니다.

5단계. API 활성화 및 확인

필요한 네 가지 API를 활성화한 뒤 다음 명령으로 현재 활성화된 API 목록을 확인합니다.

```
gcloud services list --enabled
```

컴퓨터에 설치할 수 없는 환경이라면 Google Cloud 콘솔에서 Cloud Shell을 열어 명령어를 실행할 수도 있습니다. Cloud Shell에는 `gcloud`가 준비되어 있지만, 실습용 파일은 해당 환경에 별도로 내려받아야 할 수 있습니다.

7. 비개발자 창업가 관점에서 알아둘 점

이번 강의는 단순한 설치 실습이 아닙니다. 앞으로 AI 서비스를 직접 만들고 운영할 때 필요한 기본적인 클라우드 관리 능력을 익히는 과정입니다.

특히 다음 세 가지를 기억해 두시면 좋겠습니다.

- 비용 관리: 무료 체험 크레딧과 서비스별 과금 조건을 확인하고, 실습이 끝난 리소스는 정리합니다.
- 권한 관리: 로그인 계정과 서비스 계정의 역할을 구분하고 필요한 권한만 부여합니다.
- 작업 대상 확인: 명령을 실행하기 전에 프로젝트 ID가 올바른지 확인합니다.

강의에서 설명한 무료 체험의 크레딧, 기간, 종료 후 리소스 처리 조건은 정책에 따라 바뀔 수 있습니다. 실제 가입 시에는 공식 안내를 확인하고, 무료 체험이 모든 서비스의 모든 비용을 무조건 보장한다고 생각하지 않는 것이 안전합니다.

8. 객관식 퀴즈 정답

문제: 프로젝트 이름은 `friend-agent-course`이고, 프로젝트 ID는 `friend-agent-course-471203`입니다. `--project` 또는 `gcloud config set project`에 넣어야 하는 값은 무엇인가요?

정답: `friend-agent-course-471203`

프로젝트 이름이 아니라 프로젝트 ID를 입력해야 합니다.

프로젝트 ID 예시 복사

9. 반드시 기억할 내용

- Google Cloud는 컴퓨팅 자원과 여러 서비스를 빌려 쓰는 클라우드 플랫폼입니다.
- 프로젝트는 서비스와 설정을 관리하는 기본 작업 공간입니다.
- 프로젝트 이름과 ID는 다르며, 명령어에는 일반적으로 프로젝트 ID를 사용합니다.
- `gcloud CLI`를 설치하고 로그인한 뒤 기본 프로젝트와 리전을 설정해야 합니다.
- Cloud Run, Cloud Build, Artifact Registry, Secret Manager API를 활성화해야 다음 배포 실습을 진행할 수 있습니다.
- 비용, 권한, 프로젝트 대상을 확인하는 습관이 안전한 서비스 운영의 출발점입니다.

한 문장 정리

Google Cloud에 AI 에이전트를 배포하려면 먼저 프로젝트라는 작업 공간을 만들고, `gcloud CLI`를 연결한 뒤, 필요한 API와 권한을 준비해야 합니다.

Chapter04

강의정리 챕터 04. 배포 명령 하나가 하는 일

1. 오늘 강의의 목적

이번 강의의 목적은 `gcloud run deploy --source .`라는 명령어 하나가 실제로 어떤 일을 하는지 이해하고, 간단한 웹 서비스를 직접 배포해 보는 것입니다.

앞선 강의에서 Google Cloud 프로젝트를 만들고 필요한 API를 활성화했습니다. 이제 실제로 프로그램을 올려 인터넷에서 접속할 수 있도록 만듭니다.

오늘은 처음부터 복잡한 AI 에이전트를 배포하지 않습니다. 대신 화면 한 장과 상태 확인 기능만 있는 `hello` 앱을 먼저 배포합니다. 이렇게 하면 문제가 생겼을 때 코드 문제인지, 배포 문제인지 구분하기 쉽습니다.

오늘의 학습 목표는 세 가지입니다.

- 배포 명령 한 줄 뒤에서 어떤 과정이 진행되는지 이해합니다.
- 간단한 FastAPI 앱을 Cloud Run에 배포합니다.
- 콘솔에서 빌드, 이미지, 리비전, 인스턴스, 요청 로그를 확인합니다.

2. 중학생도 이해할 수 있는 설명: 배포 명령 하나의 비밀

우리가 실행할 명령은 다음과 같습니다.

```
gcloud run deploy --source .
```

쉽게 말하면, 현재 폴더의 프로그램을 Google Cloud에 보내서 실행하고, 다른 사람이 접속할 수 있는 주소까지 만들어 달라는 뜻입니다.

이 한 줄 안에는 여러 단계가 숨어 있습니다.

1. 소스 코드

내가 작성한 프로그램과 설정 파일

2. 빌드(Build)

Cloud Build가 실행에 필요한 환경을 구성

3. 컨테이너 이미지

실행에 필요한 구성 요소를 패키지로 저장

4. 리비전(Revision)

이미지와 서비스 설정을 묶은 배포 버전

5. 실행 인스턴스와 서비스 URL

프로그램이 요청을 처리하고 사용자가 접속

이 과정을 기억하는 것이 오늘 강의의 핵심입니다.

3. 배포 과정에서 등장하는 개념 5가지

개념	쉬운 설명	담당하거나 관련된 서비스
빌드	프로그램을 실행 가능한 형태로 준비하는 작업	Cloud Build
컨테이너 이미지	실행에 필요한 요소를 묶은 패키지	Artifact Registry에 저장
리비전	특정 이미지와 설정을 기록한 배포 버전	Cloud Run
인스턴스	실제로 프로그램을 실행하는 단위	Cloud Run
서비스 URL	사용자가 접속하는 주소	Cloud Run

① 빌드와 buildpacks

빌드는 프로그램을 실행할 수 있도록 준비하는 과정입니다.

이번 실습에서는 `buildpacks`라는 도구가 소스 코드와 설정 파일을 살펴보고 Python 앱에 필요한 실행 환경과 라이브러리를 준비합니다.

예를 들어 `requirements.txt`에 다음과 같이 적혀 있다면,

```
fastapi>=0.115
uvicorn>=0.30
```

빌드 과정에서 FastAPI와 Uvicorn을 설치하도록 구성할 수 있습니다.

따라서 파일 이름이나 위치, 실행 설정이 잘못되면 빌드가 실패하거나 빌드는 성공해도 앱이 실행되지 않을 수 있습니다.

② 리비전과 인스턴스는 다릅니다

이 두 개념은 꼭 구분해야 합니다.

- 리비전: 프로그램을 어떤 이미지와 설정으로 실행할 것인지 기록한 버전입니다.
- 인스턴스: 그 리비전을 바탕으로 실제 실행되어 요청을 처리하는 단위입니다.

비유하면 리비전은 요리법이 적힌 레시피이고, 인스턴스는 그 레시피로 실제 요리를 만드는 주방입니다.

프로그램을 수정하고 다시 배포하면 새로운 리비전이 생길 수 있습니다. 요청이 많아지면 같은 리비전을 실행하는 인스턴스가 여러 개 생길 수도 있습니다.

③ 콜드 스타트(Cold Start)

요청이 없어 실행 중인 인스턴스가 0개인 상태에서 새로운 요청이 들어오면, Cloud Run은 인스턴스를 새로 시작해야 할 수 있습니다.

이때 프로그램이 시작되기까지 첫 응답이 늦어지는 현상을 콜드 스타트라고 합니다.

서버리스는 사용량에 따라 실행 자원을 줄일 수 있다는 장점이 있지만, 이런 초기 지연이 발생할 수 있다는 점도 알아야 합니다.

4. 직접 배포할 hello 앱 만들기

처음부터 복잡한 AI 에이전트를 올리면 오류 원인을 찾기 어렵습니다. 따라서 이번에는 기능이 두 개뿐인 간단한 웹 앱을 만듭니다.

- `/` : 환영 메시지를 보여 줍니다.
- `/health` : 앱이 정상적으로 응답하는지 확인합니다.

먼저 실습 저장소를 내려받고, 프로젝트 설정이 올바른지 확인합니다.

```
gh repo clone <내 GitHub 계정>/<저장소 이름>
cd <저장소 이름>
gcloud config list
```

`gh`가 설치되어 있지 않다면 `git clone`을 사용할 수 있습니다. 템플릿에서 만든 본인 저장소의 주소를 사용하시면 됩니다.

그다음 저장소 폴더와 같은 위치에 `my-hello` 폴더를 만들고 파일 네 개를 준비합니다.

my-hello/

main.py

웹 화면과 상태 확인 기능

requirements.txt

설치할 Python 라이브러리

Procfile

앱을 시작하는 명령

.python-version

사용할 Python 버전

파일 1. main.py

```
from fastapi import FastAPIfrom fastapi.responses import HTMLResponseapp = FastAPI()@a
pp.get("/", response_class=HTMLResponse)def home():    return "<h1>안녕하세요, 첫 배포입
니다</h1>"@app.get("/health")def health():    return {"status": "ok"}
```

파일 2. requirements.txt

```
fastapi>=0.115
uvicorn>=0.30
```

파일 3. Procfile

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

여기서 `$PORT`는 실행 환경이 제공하는 포트 번호를 사용한다는 뜻입니다. Cloud Run에서는 컨테이너가 지정된 포트에서 요청을 받을 수 있어야 합니다.

파일 4. .python-version

```
3.13
```

강의 교안에 제시된 Python 버전을 사용합니다. 실제 배포 시에는 해당 시점에 Cloud Run의 소스 배포 빌드 환경에서 지원하는 버전인지 확인해야 합니다.

5. hello 앱 배포하기

네 파일을 준비했다면 터미널에서 `my-hello` 폴더로 이동합니다.

```
cd my-hello
```

이제 다음 명령을 실행합니다.

```
gcloud run deploy hello \
  --source . \
  --region us-central1 \
  --allow-unauthenticated
```

명령어의 의미는 다음과 같습니다.

옵션	의미
`hello`	Cloud Run 서비스 이름
`--source .`	현재 폴더의 소스를 사용해 빌드하고 배포
`--region us-central1`	미국 중부 리전에 배포
`--allow-unauthenticated`	로그인하지 않은 사용자도 서비스 호출 가능

처음 배포할 때 컨테이너 이미지를 저장할 저장소를 만들 것인지 묻는다면, 실습 목적에 맞는 저장소인지 확인하고 진행합니다.

배포가 성공하면 터미널에 `Service URL`이 표시됩니다. 해당 주소를 브라우저에서 열어 환영 메시지가 나오는지 확인하고, 주소 뒤에 `/health`를 붙여 다음 결과가 나오는지 확인합니다.

```
{"status":"ok"}
```

주의할 점은 `--allow-unauthenticated` 옵션으로 누구나 접속할 수 있게 했다는 것입니다. 이번 앱은 간단한 연습용이지만, 실제 AI 에이전트에 적용하면 무단 호출과 LLM API 비용이 발생할 수 있으므로 인증과 사용량 제한을 별도로 설계해야 합니다.

6. 콘솔에서 배포의 다섯 단계를 확인하기

배포 명령이 성공했다면 콘솔에서 실제로 어떤 일이 있었는지 확인합니다.

1. Cloud Build — 빌드 기록

   Google Cloud 콘솔에서 Cloud Build를 열고 빌드 기록(History)을 확인합니다. 방금 실행한 배포의 빌드가 성공했는지 살펴봅니다.

2. Artifact Registry — 컨테이너 이미지

   `cloud-run-source-deploy` 저장소 안에서 `hello` 관련 이미지를 확인합니다. 배포에 사용된 이미지가 저장되는 곳입니다.

3. Cloud Run — 리비전

   Cloud Run 서비스 목록에서 `hello`를 열고 리비전(Revisions) 탭을 확인합니다. 배포 버전과 트래픽 설정을 살펴봅니다.

4. Cloud Run — 인스턴스

   측정항목(Metrics) 탭에서 컨테이너 인스턴스 수(Container instance count)를 확인합니다. 요청이 들어온 뒤 인스턴스가 실행되었는지 살펴봅니다.

5. Cloud Run — 요청 로그

   로그(Logs) 탭에서 `GET 200 /health` 같은 기록을 찾아봅니다. 브라우저에서 요청을 보낸 결과가 로그에 남았는지 확인합니다.

이 다섯 화면은 앞으로 실제 AI 에이전트에 오류가 생겼을 때도 중요한 점검 지점이 됩니다.

- 빌드가 실패했다면 빌드 로그를 봅니다.
- 앱이 시작되지 않는다면 이미지와 실행 설정, 리비전을 확인합니다.
- 요청이 들어오지 않는다면 URL과 접근 권한을 확인합니다.
- 요청이 실패했다면 서비스 로그에서 오류 메시지를 찾습니다.

7. 나의 AI 프로젝트에 적용하기

이번 실습은 MSM(My-Story-Maker)이나 ASP(AI Story Partner)를 실제 사용자에게 공개하기 전에 반드시 거쳐야 할 기초 과정입니다.

MSM을 예로 들면, 다음과 같이 단계적으로 적용할 수 있습니다.

1

기본 웹 서비스 배포

먼저 `hello` 앱으로 공개 주소를 만들고 Cloud Run이 정상적으로 동작하는지 확인합니다.

2

MSM의 최소 기능 배포

사용자가 글을 입력하고 AI가 문체를 분석하는 기능부터 연결합니다. API 키는 코드에 직접 넣지 않고 안전하게 관리합니다.

3

실행 기록과 비용 점검

실제 사용자의 요청이 정상 처리되는지 확인하고, 오류와 응답 시간, 모델 호출 비용을 측정합니다.

이런 방식으로 작은 기능부터 공개하고 검증하면 문제가 생겼을 때 원인을 좁히기 쉽습니다. 이것은 단순한 개발 요령을 넘어, 1인 AI 사업에서 시간과 비용을 아끼는 중요한 방법입니다.

8. 주관식 퀴즈 모범 답안

질문: hello 앱을 배포했는데 서비스 URL을 열어도 화면이 나오지 않는다면, 콘솔의 어느 화면을 어떤 순서로 열어 원인을 찾을까요?

다음과 같이 답할 수 있습니다.

> 먼저 Cloud Build의 빌드 기록을 열어 빌드가 성공했는지 확인하겠습니다. 빌드가 실패했다면 로그에서 Python 버전, 라이브러리 설치, 설정 파일 등의 오류를 찾겠습니다.
>
> 빌드가 성공했다면 Artifact Registry에서 컨테이너 이미지가 생성되었는지 확인하고, Cloud Run의 `hello` 서비스에서 리비전 탭을 열어 배포 버전과 트래픽 설정을 확인하겠습니다.
>
> 다음으로 측정항목 탭에서 인스턴스가 실행되었는지 살펴보고, 로그 탭에서 요청과 오류 기록을 확인하겠습니다. `/health` 주소에도 접속해 보고, 서비스 URL과 접근 권한이 올바른지도 점검하겠습니다.
>
> 이 과정을 통해 빌드 문제인지, 앱 실행 문제인지, 요청 처리 문제인지 단계적으로 구분하겠습니다.

9. 반드시 기억할 내용

- `gcloud run deploy -source .`는 소스 빌드부터 서비스 배포까지 여러 단계를 처리합니다.
- Cloud Build는 빌드를 수행하고, Artifact Registry는 이미지를 저장합니다.

- Cloud Run의 리비전은 배포 버전이고, 인스턴스는 실제로 요청을 처리하는 실행 단위입니다.
- 요청이 없으면 인스턴스가 0개로 줄어들 수 있으며, 첫 요청에는 콜드 스타트가 발생할 수 있습니다.
- 문제가 생기면 빌드 기록 → 이미지 → 리비전 → 인스턴스 → 요청 로그 순서로 확인하면 원인을 좁히기 쉽습니다.
- 누구나 호출할 수 있도록 공개한 AI 서비스는 인증, 사용량 제한, API 비용 관리가 필요합니다.

한 문장 정리

배포 명령 한 줄 뒤에는 빌드, 이미지 생성, 리비전 생성, 인스턴스 실행이라는 과정이 있으며, 이 흐름을 이해하면 AI 서비스를 직접 배포하고 오류를 진단할 수 있습니다.

Chapter05

/강의정리 챕터 05. 같은 일을 이번에는 혼자

1. 이번 강의의 목적

이번 강의의 핵심은 Cloud Run에 배포하는 방법을 혼자 반복하고, 문제가 생겼을 때 이전 버전으로 되돌리는 방법까지 익히는 것입니다.

앞선 챕터에서는 코드를 클라우드에 배포해 서비스 URL을 만드는 과정을 배웠습니다. 이번에는 한 단계 더 나아가 실제 서비스 운영에 필요한 세 가지 기능을 연습합니다.

- 리비전(Revision): 배포할 때마다 기록되는 서비스 버전
- 롤백(Rollback): 문제가 생겼을 때 이전 버전으로 되돌리는 것
- 트래픽 분할(Traffic Splitting): 여러 버전에 사용자 요청을 나누어 보내는 것

쉽게 말하면, 이제는 서비스를 만드는 것뿐 아니라 새 버전을 안전하게 시험하고, 문제가 생기면 복구하는 방법까지 배우는 것입니다.

2. 중학생도 이해할 수 있는 설명

2-1. 리비전은 서비스의 버전 기록입니다

처음에 화면에 `안녕하세요`라고 표시되는 서비스를 배포했다고 가정하겠습니다.

리비전 1

첫 번째 배포

화면: 안녕하세요

이전 버전

리비전 2

두 번째 배포

화면: 두 번째 배포입니다

현재 서비스 버전

화면 문구를 바꾸고 다시 배포하면 두 번째 리비전이 만들어집니다. 첫 번째 리비전은 사라지지 않고 남아 있습니다.

중요한 점은 코드가 바뀌지 않더라도 서비스 설정이 바뀌면 새 리비전이 만들어질 수 있다는 것입니다. 리비전에는 실행할 코드뿐 아니라 서비스 설정도 함께 반영되기 때문입니다.

2-2. 롤백은 이전 버전으로 돌아가는 것입니다

새 버전에서 오류가 발견되었다고 가정하겠습니다.

이때 코드를 다시 수정하고 재배포하는 방법도 있지만, 기존에 정상 작동했던 리비전으로 트래픽을 돌리면 빠르게 복구할 수 있습니다.

롤백 전

리비전 1

이전 버전

리비전 2

트래픽 100%

트래픽 전환

롤백 후

리비전 1

트래픽 100%

리비전 2

트래픽 0%

여기서 중요한 사실이 하나 더 있습니다. 롤백은 코드를 삭제하거나 리비전을 지우는 것이 아니라, 사용자 요청이 전달되는 대상을 바꾸는 것입니다.

서비스 URL도 일반적으로 그대로 유지됩니다. 사용자는 같은 주소로 접속하지만, Cloud Run이 이전 버전으로 요청을 전달합니다.

2-3. 트래픽 분할은 새 버전을 시험하는 방법입니다

새 버전을 모든 사용자에게 한 번에 공개하는 대신, 기존 버전과 새 버전에 요청을 절반씩 나눠 보낼 수 있습니다.

리비전 1 · 기존 버전

50%

리비전 2 · 새 버전

50%

예시: 각 요청이 어느 버전으로 전달될지는 Cloud Run이 결정합니다.

열 번 요청했다고 정확히 다섯 번씩 나오는 것은 아닙니다. 예를 들어 기존 버전이 6번, 새 버전이 4번 나와도 정상일 수 있습니다. 요청을 많이 보낼수록 전체 결과가 설정한 비율에 가까워질 가능성이 높아집니다.

이런 방식으로 새 버전을 일부 트래픽에만 먼저 노출하는 것을 카나리 배포(Canary Deployment) 라고 합니다.

3. 직접 실습하는 순서

이번 실습은 다섯 단계로 진행됩니다. 핵심은 명령어를 외우는 것보다 실행한 뒤 무엇을 확인해야 하는지 이해하는 것입니다.

1. 화면 문구 변경하기

   `my-hello/main.py`에서 화면에 표시되는 문구를 `두 번째 배포입니다`로 바꿉니다.

2. 다시 배포하기

   프로젝트 폴더에서 다음 명령어를 실행합니다.

   `bash gcloud run deploy hello --source . --region us-central1 --allow-unauthenticated`

   배포가 끝나면 기존 서비스 URL에 접속해 문구가 바뀌었는지 확인합니다.

3. 리비전 목록 확인하기

   `bash gcloud run revisions list --service hello --region us-central1`

   리비전 이름, 트래픽 비율, 생성 시각을 확인합니다. 두 번째 배포까지 정상적으로 완료되었다면 보통 `hello-00001`, `hello-00002`와 같은 리비전이 보입니다. 실제 이름은 명령

어 결과를 기준으로 확인하세요.

4. 첫 번째 리비전으로 롤백하기

   첫 번째 리비전의 정확한 이름을 목록에서 복사한 뒤 실행합니다.

   `bash gcloud run services update-traffic hello --region us-central1 --to-revisions <첫_리비전_이름>=100`

   브라우저를 새로고침해 처음 문구가 표시되는지 확인합니다.

5. 최신 리비전으로 복구하기

   `bash gcloud run services update-traffic hello --region us-central1 --to-latest`

   새 버전이 다시 트래픽을 받는지 확인합니다. 이 단계까지 마쳐야 실습이 완료됩니다.

주의할 점은 `<첫_리비전_이름>`을 그대로 입력하는 것이 아니라 실제 리비전 이름으로 바꾸는 것입니다. 또한 `--to-latest`는 가장 최근 리비전으로 트래픽을 돌리는 명령입니다.

4. 트래픽을 50:50으로 나누는 추가 실습

두 리비전에 트래픽을 반씩 보내면 두 버전의 결과를 비교할 수 있습니다.

```
gcloud run services update-traffic hello --region us-central1 --to-revisions <첫_리비전_이름>=50,<둘째_리비전_이름>=50
```

그다음 서비스 URL에 열 번 요청해 결과를 비교합니다.

Windows PowerShell에서는 다음과 같이 실행할 수 있습니다.

```
1..10 | ForEach-Object {
  (Invoke-WebRequest -UseBasicParsing <공개_서비스_URL>).Content
}
```

`<공개_서비스_URL>`은 실제 서비스 주소로 바꿔 주세요. URL을 여러 번 호출하면 두 문구가 섞여 나타날 수 있습니다.

실험이 끝나면 반드시 다음 명령어로 최신 리비전으로 돌려놓습니다.

```
gcloud run services update-traffic hello --region us-central1 --to-latest
```

5. 코딩 에이전트에게 맡길 때의 원칙

이번 실습은 AI 에이전트에게 명령어를 맡길 때도 중요한 원칙을 보여 줍니다.

서비스에 영향을 주는 명령은 실행 전에 확인하고 승인하는 것이 안전합니다.

예를 들어 에이전트에게 다음과 같이 요청할 수 있습니다.

> `hello` 서비스의 리비전 목록과 현재 트래픽을 받는 리비전을 보여 주세요. 첫 번째 리비전으로 트래픽 100%를 전환하는 명령을 실행 전에 먼저 제시해 주세요. 제가 명령어와 리비전 이름을 확인하고 승인하면 실행해 주세요.

승인 전에 확인할 사항은 세 가지입니다.

- 서비스 이름이 `hello`인지
- 전환 대상 리비전이 의도한 버전인지
- 트래픽 비율이 의도대로 100%인지

이 원칙은 사용자가 배우고 있는 사람이 승인하는 AI 에이전트(HITL) 와도 연결됩니다. AI가 모든 일을 자동으로 처리하게 하기보다, 영향이 큰 작업에서는 실행 계획을 먼저 보여 주고 사람이 승인하도록 설계하는 것입니다.

6. 내 프로젝트에 적용한다면?

현재 진행 중인 MSM(My-Story-Maker)과 ASP(AI Story Partner)에 다음과 같이 적용할 수 있습니다.

적용 프로젝트	리비전과 롤백 활용
MSM	AI가 원문을 다듬는 기능을 개선했을 때 새 버전을 배포하고, 결과가 나빠지면 이전 버전으로 복구
ASP	AI 상담·컨설팅 서비스의 새 기능을 일부 사용자에게 먼저 공개하고 결과 확인
뉴스레터 에이전트	뉴스 수집·선별·요약 로직을 개선한 뒤 새 버전의 결과를 기존 버전과 비교

특히 MSM은 단순히 화면이 잘 나오는지만 확인해서는 부족합니다. 예를 들어 문체 분석 기능을 개선했다면 다음과 같은 기준을 함께 비교해야 합니다.

- 원문의 말투와 개성이 잘 유지되는가?
- AI가 원문에 없는 사실을 추가하지 않는가?
- 사용자가 수정 결과를 이해하고 원하는 대로 조정할 수 있는가?

새 버전이 화면에서는 정상적으로 작동하더라도 결과 품질이 나빠질 수 있기 때문입니다. 따라서 배포 성공 여부와 서비스 품질은 별도로 검증해야 합니다.

7. 객관식 퀴즈 정답

정답: 2번

새 리비전이 하나 생깁니다. 리비전에는 이미지뿐 아니라 설정도 함께 기록되기 때문입니다.

코드가 그대로여도 요청 타임아웃 같은 서비스 설정을 변경하면 새로운 리비전이 만들어질 수 있습니다. 다만 새 리비전이 생겼다는 사실만으로 기존 리비전의 트래픽이 자동으로 모두 새 리비전으로 이동한다고 단정해서는 안 됩니다. 트래픽 할당 상태도 별도로 확인해야 합니다.

8. 핵심 요약 및 반드시 기억할 내용

- 리비전: 배포된 서비스 버전의 기록입니다.
- 롤백: 이전 리비전으로 트래픽을 전환해 서비스를 복구하는 방법입니다.
- 트래픽 분할: 여러 리비전에 요청을 나누어 보내는 기능입니다.
- 카나리 배포: 새 버전을 일부 사용자에게 먼저 공개해 안전성을 확인하는 방식입니다.
- 운영 원칙: 배포 후에는 화면뿐 아니라 리비전과 트래픽 비율도 확인해야 합니다.
- 에이전트 안전 원칙: 서비스에 영향을 주는 명령은 실행 전에 확인하고 승인하는 절차를 둘 수 있습니다.

이번 강의에서 가장 중요한 것은 명령어 자체가 아닙니다. 새 버전을 배포하고, 확인하고, 문제가 있으면 복구할 수 있는 운영 능력입니다.

한 문장 정리

클라우드 서비스 운영은 새 버전을 배포하는 것으로 끝나는 것이 아니라, 버전을 관리하고 트래픽을 조절하며 문제가 생겼을 때 안전하게 되돌릴 수 있어야 완성됩니다.

Chapter06

/강의정리 챕터 06. 배포 전 로컬 실행

1. 이번 강의의 목적

이번 강의의 핵심은 클라우드에 배포하기 전에 내 컴퓨터에서 서비스를 먼저 실행하고, API 키를 안전하게 관리하는 방법을 익히는 것입니다.

앞선 챕터에서는 Cloud Run에 서비스를 배포하고, 리비전을 관리하며, 문제가 생겼을 때 이전 버전으로 되돌리는 방법을 배웠습니다. 이번에는 그보다 앞 단계인 배포 전 점검을 연습합니다.

이번 강의에서 반드시 이해해야 할 내용은 세 가지입니다.

- 로컬 실행: 내 컴퓨터에서 서비스가 정상적으로 작동하는지 확인합니다.
- 환경변수와 `.env`: API 키와 같은 민감한 정보를 코드와 분리해 관리합니다.
- 배포 환경의 차이: 내 컴퓨터에서 사용하던 API 키가 클라우드 서버에 자동으로 전달되는 것은 아닙니다.

2. 중학생도 이해할 수 있는 설명

2-1. 로컬 실행은 가게를 열기 전 리허설입니다

식당을 새로 열었다고 생각해 보겠습니다. 손님을 받기 전에 주방에서 음식을 직접 만들어 보고, 주문이 들어왔을 때 제대로 나오는지 확인해야 합니다.

AI 서비스도 같습니다.

내 컴퓨터

1단계. 로컬 실행

화면이 열리는가? 캐릭터를 저장할 수 있는가? AI가 답하는가?

Cloud Run

2단계. 클라우드 배포

서버에서도 같은 기능이 작동하는가?

로컬에서는 정상인데 서버에서만 실패한다면 두 환경의 차이를 비교할 수 있습니다.

예를 들어 다음과 같은 원인을 구분할 수 있습니다.

- 로컬에서도 실패한다면 코드, API 키, 패키지 설치 등을 점검합니다.
- 로컬에서는 성공하지만 서버에서 실패한다면 서버의 환경변수, 권한, 네트워크, 실행 설정 등을 점검합니다.

따라서 로컬 실행은 단순한 연습이 아니라 문제의 원인을 좁히는 첫 번째 진단 단계입니다.

2-2. `.env` 파일은 비밀번호를 보관하는 개인 금고입니다

AI 서비스는 외부 LLM에 요청을 보낼 때 API 키가 필요할 수 있습니다. 이 키는 서비스 이용 권한과 비용에 연결되므로 안전하게 보관해야 합니다.

`.env` 파일은 이런 설정값을 코드와 분리해 보관하는 파일입니다.

예를 들어 다음과 같은 설정이 들어갑니다.

```
LLM_PROVIDER=openai
LLM_API_KEY=여기에_발급받은_API_키
```

- `LLM_PROVIDER`: 어떤 AI 제공사를 사용할지 지정합니다.
- `LLM_API_KEY`: 해당 제공사의 API를 호출할 때 사용하는 키입니다.

강의 실습에서는 제공받은 키에 맞춰 `openai` 또는 `gemini`를 지정합니다. 키가 아직 준비되지 않았다면 임의의 값을 넣어 실제 AI 호출이 된다고 생각해서는 안 됩니다.

여기서 중요한 보안 원칙이 있습니다.

API 키를 코드에 직접 넣거나 AI 대화창에 붙여 넣지 마세요. 대화 기록이나 코드 저장소에 키가 남을 수 있기 때문입니다. 키는 로컬 편집기에서 직접 입력하고, 외부에 노출되지 않도록 관리해야 합니다.

3. 실습 순서: 로컬에서 실행하기

아래는 강의 내용을 실제 작업 순서대로 정리한 것입니다.

1단계. 가상환경 만들기

저장소 루트 폴더에서 실행합니다.

```
python -m venv .venv
```

가상환경은 프로젝트에 필요한 파이썬 패키지를 다른 프로젝트와 분리해 관리하는 공간입니다.

가상환경을 활성화한 뒤 필요한 패키지를 설치합니다.

```
# macOS 또는 Linux
source .venv/bin/activate

# Windows
.venv\Scripts\activate

# 패키지 설치
pip install -r requirements.txt
```

쉽게 말하면 가상환경은 프로젝트마다 따로 마련하는 작업 도구 상자입니다. 프로젝트 A에 설치한 패키지가 프로젝트 B에 영향을 주는 일을 줄여 줍니다.

2단계. `.env` 파일 만들기

먼저 예제 파일을 복사합니다.

```
# macOS 또는 Linux
cp .env.example .env

# Windows
copy .env.example .env
```

그다음 텍스트 편집기로 `.env`를 직접 열어 다음 항목을 설정합니다.

```
LLM_PROVIDER=openai
LLM_API_KEY=발급받은_실제_API_키
```

Gemini 키를 사용하는 경우에는 `LLM_PROVIDER=gemini`로 지정합니다. 실제로 어떤 제공사를 지원하는지는 프로젝트 코드와 설정을 기준으로 확인해야 합니다.

키를 저장한 뒤 앱을 실행했는데 `LLM_API_KEY`가 없다는 오류가 나온다면 다음을 확인합니다.

- `.env` 파일을 저장했는가?
- 파일 이름이 `.env`인가? `.env.txt`로 저장되지는 않았는가?
- 키 앞뒤에 불필요한 공백이나 따옴표가 들어가지는 않았는가?
- 앱이 `.env`를 실제로 읽도록 구성되어 있는가?

마지막 항목도 중요합니다. `.env` 파일이 존재한다고 해서 파이썬이 자동으로 읽는 것은 아닙니다. 프로젝트의 설정 코드가 해당 파일을 읽도록 되어 있어야 합니다.

3단계. 앱 실행하기

```
uvicorn app.main:app --port 8080
```

실행이 정상적으로 되면 브라우저에서 다음 주소를 엽니다.

http://localhost:8080

이 화면에서 캐릭터를 만들고 저장한 다음, AI에게 몇 마디 말을 걸어 보세요.

특히 다음 세 가지를 확인합니다.

- 캐릭터가 정상적으로 저장되는가?
- AI가 실제로 응답하는가?
- 오른쪽 `이번 턴` 패널에 사용한 도구와 소요 시간이 표시되는가?

소요 시간은 나중에 Cloud Run의 요청 타임아웃을 설정할 때 참고할 수 있습니다. 다만 한 번 측정한 시간만으로 타임아웃을 결정하기보다는 여러 번 실행해 응답 시간의 범위를 확인하는 것이 좋습니다.

실행을 끝낼 때는 앱이 실행 중인 터미널에서 `Ctrl+C`를 누릅니다.

4. `.env` 파일은 어디까지 전달될까요?

이번 강의에서 가장 중요한 보안 개념입니다.

로컬에서 사용한 `.env`가 Git이나 클라우드 배포 과정에 포함되지 않도록 설정되어 있어야 합니다.

내 컴퓨터의 `.env`

API 키가 저장된 로컬 설정 파일

로컬 보관

Git 저장소

`.gitignore`에서 `.env`를 제외하도록 설정

제외

Cloud Run 배포

`.gcloudignore`에서 `.env`를 제외하도록 설정

제외

두 설정 파일의 역할은 다릅니다.

파일	역할
`.gitignore`	Git이 추적하지 않을 파일을 지정합니다.
`.gcloudignore`	`gcloud run deploy --source .`로 소스를 올릴 때 제외할 파일을 지정합니다.
`.env`	로컬에서 사용하는 환경설정과 비밀값을 보관합니다.

다음 명령어로 제외 설정을 확인할 수 있습니다.

```
git status
grep -n "^.env$" .gitignore .gcloudignore
```

Windows PowerShell에서는 다음과 같이 실행합니다.

```
Select-String -Pattern "^.env$" .gitignore, .gcloudignore
```

`git status`에 `.env`가 보이지 않는 것은 일반적으로 Git이 이 파일을 추적하지 않고 있다는 뜻입니다. 이것이 키가 GitHub에 저장되었다는 의미는 아닙니다. 다만 이미 Git에서 추적 중인 파일은 `.gitignore`에 추가하는 것만으로 추적이 해제되지 않으므로 별도로 확인해야 합니다.

또한 `.gcloudignore`에서 `.env`를 제외했다면 Cloud Run 서버에 로컬 키가 자동으로 전달되지 않습니다.

서버에서 AI 기능을 사용하려면 Cloud Run에도 API 키를 별도로 안전하게 설정해야 합니다. 실제 운영에서는 Secret Manager를 이용해 비밀값을 관리하는 방식을 고려할 수 있습니다.

5. 코딩 에이전트에게 맡길 때의 원칙

가상환경을 만들고 패키지를 설치하는 작업은 코딩 에이전트에게 맡길 수 있습니다. 다만 API 키는 직접 입력하도록 하는 것이 안전합니다.

다음은 강의에서 제시한 요청의 핵심입니다.

> 저장소 루트에 가상환경을 만들고 `requirements.txt`를 설치해 주세요. 그다음 `.env.example`을 `.env`로 복사만 해 주세요. 키는 제가 직접 입력할 것이므로 `.env`의 내용을 읽거나 출력하지 마세요.

이 요청에서 배울 점은 작업을 맡기되 민감한 정보에 접근하지 못하도록 범위를 제한한다는 것입니다.

이 원칙은 이전 챕터에서 배운 사람이 승인하는 에이전트와도 연결됩니다. AI에게 모든 권한을 주는 대신, 허용할 작업과 제한할 작업을 구분하는 것입니다.

6. 내 프로젝트에 적용한다면?

현재 진행 중인 프로젝트에서도 같은 방식으로 활용할 수 있습니다.

프로젝트	로컬에서 먼저 확인할 기능	서버에서 별도로 설정할 것
MSM	원문 입력, 문체 분석, 글 다듬기	LLM API 키, 서비스 환경변수
ASP	AI 컨설팅 요청, 답변 생성	운영용 API 키, 권한 및 배포 설정
뉴스레터 에이전트	수집·선별·요약·검증 파이프라인	API 키, 예약 실행 설정, 로그

예를 들어 MSM에서 원문을 입력하면 AI가 문체를 분석하는 기능을 만들었다고 가정하겠습니다.

먼저 내 컴퓨터에서 원문 입력부터 분석 결과 출력까지 확인합니다. 이때 정상 작동한다면 그다음 클라우드에 배포합니다. 서버에서만 실패한다면 API 키가 전달되었는지, 필요한 환경변수가 설정되었는지부터 점검할 수 있습니다.

이런 순서는 개발 경험이 많지 않아도 문제의 원인을 단계적으로 좁힐 수 있게 해 줍니다.

7. 객관식 퀴즈 정답

정답: 2번

`.gcloudignore`가 `.env`를 제외하도록 설정되어 있다면, 서버에는 로컬 API 키가 자동으로 전달되지 않습니다. 서버에 키를 별도로 설정해야 합니다.

Cloud Run에서 앱을 실행하려면 서버 환경에도 필요한 환경변수가 있어야 합니다. 운영 환경에서는 Secret Manager 등을 통해 키를 안전하게 전달하는 방식을 사용할 수 있습니다.

8. 핵심 요약 및 반드시 기억할 내용

- 배포 전에 로컬에서 실행합니다. 문제를 코드 문제와 배포 환경 문제로 구분하는 데 도움이 됩니다.
- 가상환경과 패키지를 준비합니다. `.venv`를 만들고 `requirements.txt`를 설치합니다.
- API 키는 `.env`에 보관합니다. 코드에 직접 넣거나 AI 대화창에 붙여 넣지 않습니다.
- `.gitignore`와 `.gcloudignore`는 역할이 다릅니다. 하나는 Git 추적을, 다른 하나는 소스 배포 시 파일 제외를 관리합니다.
- 로컬의 API 키는 서버에 자동으로 전달되지 않습니다. Cloud Run에도 필요한 비밀값을 별도로 안전하게 설정해야 합니다.
- 로컬 성공이 서버 성공을 보장하지는 않습니다. 서버의 환경변수, 권한, 실행 설정 등을 별도로 점검해야 합니다.

한 문장 정리

안전한 AI 서비스 배포는 내 컴퓨터에서 먼저 검증하고, API 키를 보호하며, 클라우드 환경에도 필요한 설정을 정확히 전달하는 것에서 시작합니다.

Chapter07

/강의정리 챕터 07. 키 없이 올라간 첫 배포

1. 이번 강의의 목적

이번 강의의 핵심은 배포 실패의 진짜 원인을 로그에서 찾고, AI API 키를 안전하게 클라우드에 연결하는 방법을 배우는 것입니다.

앞선 챕터에서는 로컬 컴퓨터에서 `.env` 파일을 이용해 AI 서비스를 실행했습니다. 그런데 이 파일은 보안을 위해 클라우드 배포 대상에서 제외했습니다. 그 결과, 내 컴퓨터에서는 잘 작동하던 서비스가 Cloud Run에서는 시작하지 못하는 상황이 발생합니다.

이번 강의에서는 이 문제를 해결하면서 다음 네 가지를 익힙니다.

1. 오류 메시지와 실제 원인을 구분하는 방법
2. Cloud Run 로그를 근거로 문제를 진단하는 방법
3. Secret Manager로 API 키를 안전하게 전달하는 방법
4. 불필요한 실습 서비스와 이미지를 안전하게 정리하는 방법

이 강의는 단순한 배포 실습을 넘어, 실제 AI 서비스를 운영할 때 필요한 보안과 장애 대응의 기본기를 다룹니다.

2. 중학생도 이해할 수 있는 설명

2-1. 포트 오류처럼 보이지만 진짜 원인은 API 키입니다

Cloud Run에 서비스를 배포하면 터미널에 다음과 같은 오류가 나타날 수 있습니다.

> 컨테이너가 PORT=8080에서 시작하지 못했습니다.

이 문구만 보면 포트 설정이 잘못된 것처럼 생각하기 쉽습니다. 하지만 이번 실습에서 실제 원인은 LLM API 키가 없어서 앱이 시작하자마자 종료된 것입니다.

1. Cloud Run에 배포

빌드는 성공

2. 앱이 API 키 확인

LLM_API_KEY가 없음

3. 앱이 시작 중 종료

포트를 열고 요청을 기다리지 못함

4. Cloud Run이 시작 실패로 판단

터미널에는 포트 오류가 표시됨

비유하면, 식당 주방에 손님을 받을 준비가 되었는지 확인했더니 조리사가 필수 재료를 받지 못해 아예 출근을 포기한 상황입니다. 겉으로는 식당 문이 열리지 않은 것이지만, 진짜 원인은 문이 아니라 재료가 없다는 것입니다.

따라서 오류 메시지에 포트가 등장한다고 무조건 포트 설정부터 고치면 안 됩니다.

오류 메시지는 어떤 문제가 나타났는지를 보여 주고, 로그는 왜 그 문제가 발생했는지 알려 줄 수 있습니다.

2-2. 로그를 근거로 원인을 찾아야 합니다

이번 실습에서 로그를 확인하는 순서는 다음과 같습니다.

1. Google Cloud 콘솔에서 Cloud Run을 엽니다.
2. `friend-agent` 서비스를 선택합니다.
3. 로그(Logs) 탭에서 심각도(Severity)를 오류(Error)로 설정합니다.
4. `LLM_API_KEY`가 없어서 앱을 시작하지 못한다는 기록을 찾습니다.
5. 해당 로그를 근거로 원인을 설명하고, 확인 전에는 임의로 수정하지 않습니다.

AI 코딩 에이전트에게 문제를 맡길 때도 같은 원칙을 적용할 수 있습니다.

> `friend-agent` 배포가 실패했습니다. 실패한 리비전의 Cloud Run 로그를 읽고, 로그에 적힌 내용을 근거로 원인을 설명해 주세요. 아직 아무것도 고치지 마세요.

이 요청에는 두 가지 중요한 조건이 들어 있습니다.

- 근거를 요구합니다. 로그를 읽고 판단하게 합니다.
- 수정을 제한합니다. 원인을 확인하기 전에 설정을 임의로 변경하지 못하게 합니다.

이렇게 하면 에이전트가 표면적인 오류 메시지만 보고 정상적인 포트 설정을 변경하는 위험을 줄일 수 있습니다.

3. API 키를 안전하게 전달하는 방법

3-1. 일반 환경변수와 Secret Manager의 차이

Cloud Run에 API 키를 전달하는 방법은 여러 가지가 있습니다. 이번 강의에서는 일반 환경변수에 키 값을 직접 넣는 방식과 Secret Manager를 이용하는 방식을 비교합니다.

방법 A. 환경변수에 키 값을 직접 입력

서비스 설정에 실제 키 값이 기록될 수 있습니다.

- 리비전 설정에 키 값이 남을 수 있습니다.

- 이전 리비전에도 과거 키 값이 남을 수 있습니다.
- 배포 명령이나 터미널 기록에 키가 노출될 수 있습니다.

방법 B. Secret Manager 사용

키 값은 별도로 보관하고 서비스에는 시크릿의 이름을 연결합니다.

- 서비스 설정에는 `llm-api-key` 같은 참조 정보가 기록됩니다.
- 읽기 권한을 가진 계정만 키를 가져갈 수 있도록 제한할 수 있습니다.
- 새 키 버전을 추가해 교체 작업을 관리할 수 있습니다.

비유하면 일반 환경변수 방식은 금고의 비밀번호를 작업 지시서에 적어 놓는 것과 비슷합니다. Secret Manager 방식은 비밀번호를 금고에 보관하고, 필요한 프로그램에 금고를 열 수 있는 권한만 주는 것과 같습니다.

운영용 API 키는 Secret Manager와 같은 비밀 관리 수단으로 보관하는 것이 좋습니다. 다만 Secret Manager를 사용한다고 모든 위험이 사라지는 것은 아닙니다. 시크릿을 읽을 권한이 있는 계정과 사용자도 적절히 제한해야 합니다.

3-2. 실습: Secret Manager에 키 저장하기

먼저 Google Cloud 콘솔에서 `Secret Manager`를 검색합니다.

1. 보안 비밀 만들기(Create secret) 를 선택합니다.
2. 이름을 `llm-api-key`로 지정합니다.
3. 로컬 `.env`에 입력한 것과 같은 실제 API 키를 보안 비밀 값에 입력합니다.
4. 생성을 완료하고 `llm-api-key`의 버전 1이 사용 설정(Enabled) 상태인지 확인합니다.

이 작업은 키 값을 콘솔에서 직접 입력하는 방식으로 진행합니다. 키를 명령어에 붙여 넣거나 코딩 에이전트 대화창에 입력하지 않습니다.

3-3. 서비스 계정에 읽기 권한 주기

서비스 계정은 사람이 아닌 프로그램이 Google Cloud에서 작업할 때 사용하는 계정입니다. Cloud Run이 실행 중 시크릿을 읽으려면 해당 런타임 서비스 계정에 권한이 필요합니다.

강의에서는 기본 Compute Engine 서비스 계정을 사용하는 예시를 제시합니다. 다만 실제 서비스가 다른 런타임 서비스 계정으로 설정되어 있다면 실제로 사용 중인 계정에 권한을 부여해야 합니다.

먼저 프로젝트 번호를 확인합니다.

```
gcloud projects describe <PROJECT_ID> --format="value(projectNumber)"
```

`<PROJECT_ID>`를 실제 프로젝트 ID로 바꿉니다.

그다음 해당 계정에 `llm-api-key` 시크릿 하나에 대한 읽기 권한을 부여합니다.

```
gcloud secrets add-iam-policy-binding llm-api-key \
  --member="serviceAccount:<프로젝트번호>-compute@developer.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
```

이 명령에서 반드시 확인할 부분은 두 가지입니다.

- 권한이 `roles/secretmanager.secretAccessor`인지
- 권한을 부여하는 대상이 프로젝트 전체가 아니라 `llm-api-key`라는 시크릿 하나인지

권한을 넓게 부여할수록 실수나 침해가 발생했을 때 피해 범위가 커질 수 있습니다. 따라서 필요한 범위에만 권한을 주는 것이 원칙입니다.

3-4. Cloud Run 서비스에 시크릿 연결하기

이제 `friend-agent`의 `LLM_API_KEY` 환경변수가 Secret Manager의 최신 버전을 참조하도록 설정합니다.

```
gcloud run services update friend-agent \
  --region us-central1 \
  --update-secrets LLM_API_KEY=llm-api-key:latest
```

Gemini 키를 사용하는 경우에는 제공사 설정도 맞춰야 합니다.

```
gcloud run services update friend-agent \
  --region us-central1 \
  --update-secrets LLM_API_KEY=llm-api-key:latest \
  --update-env-vars LLM_PROVIDER=gemini
```

이 명령은 소스 코드를 다시 빌드하는 대신 서비스 설정을 변경합니다. Cloud Run은 설정이 변경된 새 리비전을 만들 수 있습니다.

여기서 기억할 핵심은 앱 코드는 계속 `LLM_API_KEY`라는 환경변수를 읽지만, 실제 키 값은 Secret Manager에서 관리한다는 점입니다.

설정이 끝나면 다음을 확인합니다.

- `friend-agent`의 새 리비전이 생성되었는가?
- 리비전의 `Variables & Secrets`에 `LLM_API_KEY`가 `llm-api-key`의 `latest` 버전을 참조한다고 표시되는가?
- 서비스 URL에서 캐릭터를 만들고 AI 응답을 받을 수 있는가?

화면에 시크릿 참조가 표시되는 것과 실제 AI 응답이 성공하는 것은 별개입니다. 두 가지를 모두 확인해야 합니다.

4. 키 교체와 운영상 주의점

API 키가 유출되었거나 주기적으로 교체해야 한다면 Secret Manager에 새 버전을 추가할 수 있습니다.

1. `llm-api-key`에 새 버전을 추가합니다.
2. Cloud Run에서 시크릿 참조를 다시 적용해 새 리비전을 만듭니다.
3. 새 리비전에서 AI 응답이 정상적으로 오는지 확인합니다.
4. 새 키가 정상적으로 적용된 것을 확인한 뒤 불필요한 이전 키 버전을 사용 중지하거나 폐기합니다.

주의할 점은 `latest`를 참조한다고 해서 이미 실행 중인 모든 인스턴스가 새 키를 즉시 읽는 것은 아니라는 것입니다. 환경변수 방식으로 시크릿을 주입하면 인스턴스가 시작될 때 읽은 값이 사용될 수 있으므로, 새 리비전을 만들어 새 키 적용을 확인하는 것이 중요합니다.

또 하나의 한계도 있습니다. 여러 서비스가 같은 기본 서비스 계정을 사용한다면, 그 계정에 시크릿 읽기 권한을 준 결과 다른 서비스도 같은 시크릿을 읽을 수 있을 수 있습니다. 가능하면 서비스별로 런타임 서비스 계정을 분리하고, 필요한 시크릿에만 권한을 부여하는 것이 더 안전합니다.

5. 코딩 에이전트에게 맡길 때의 원칙

이번 강의에서 에이전트에게 맡길 수 있는 작업은 다음과 같습니다.

- 실패한 리비전의 로그를 읽고 원인을 설명하기
- 서비스의 런타임 서비스 계정을 확인하기
- 시크릿에 읽기 권한을 주는 명령을 제시하기
- Cloud Run의 시크릿 연결 설정을 변경하기
- 변경 후 리비전과 서비스 상태를 확인하기

다만 시크릿의 실제 값은 에이전트에게 보여 주지 않습니다. 특히 권한 변경과 서비스 설정 변경은 실행 전에 명령과 대상 범위를 확인하는 것이 좋습니다.

예를 들어 다음과 같이 요청할 수 있습니다.

> Secret Manager에 `llm-api-key`라는 시크릿을 만들어 두었습니다. 키 값은 읽거나 출력하지 마세요. `friend-agent`의 실제 런타임 서비스 계정을 확인하고, 그 계정에 `llm-api-key` 하나에 대해서만 읽기 권한을 주도록 해 주세요. 이어서 `LLM_API_KEY`가 해당 시크릿의 최신 버전을 참조하도록 서비스 설정을 변경하는 명령을 제시해 주세요. 명령마다 무엇을 바꾸는지 설명하고, 실행 전에 제 승인을 받아 주세요.

이것은 앞선 챕터의 사람이 승인하는 에이전트 원칙과 연결됩니다. AI가 작업을 준비하더라도 권한과 비밀정보가 관련된 변경은 사람이 확인하고 승인하도록 설계하는 것입니다.

6. 실습용 서비스와 이미지 정리하기

강의 마지막에는 앞서 사용했던 `hello` 서비스를 삭제합니다. 여기서 중요한 점은 Cloud Run 서비스를 삭제해도 Artifact Registry에 저장된 컨테이너 이미지는 자동으로 삭제되지 않을 수 있다는 것입니다.

서비스와 이미지는 별도로 관리해야 합니다.

Cloud Run 서비스

실행 중인 서비스와 해당 서비스의 배포 설정

Artifact Registry 이미지

서비스를 실행하기 위해 빌드한 컨테이너 이미지

실습에서 `hello` 서비스만 정리하려면 다음 순서로 진행합니다.

1단계. Cloud Run 서비스 삭제

```
gcloud run services delete hello --region us-central1
```

2단계. Artifact Registry에서 `hello` 패키지만 삭제

```
gcloud artifacts packages delete hello \
  --repository cloud-run-source-deploy \
  --location us-central1
```

3단계. 남은 패키지 확인

```
gcloud artifacts packages list \
  --repository cloud-run-source-deploy \
  --location us-central1
```

삭제 전에는 대상이 `hello`이고 리전이 `us-central1`인지 확인합니다. 저장소 `cloud-run-source-deploy` 자체를 삭제하면 안 됩니다. `friend-agent`의 이미지도 같은 저장소에 있을 수 있기 때문입니다.

또한 삭제 명령을 실행하기 전에는 현재 서비스와 이미지의 상태를 확인하고, 실제로 삭제해도 되는 대상인지 검토하세요.

7. 이번 강의의 내용을 내 프로젝트에 적용한다면?

사용자님의 주요 프로젝트인 MSM과 ASP에 적용하면 다음과 같습니다.

프로젝트	적용 방법	기대 효과
MSM	문체 분석 및 글 다듬기 기능을 로컬에서 먼저 검증하고, API 키는 Secret Manager로 관리	원문 처리 기능의 오류를 줄이고 비밀정보 보호
ASP	AI 컨설팅 서비스의 API 키와 서비스별 권한을 분리	운영 보안 강화 및 비용·권한 관리
뉴스레터 에이전트	수집·선별·요약 단계의 로그를 확인하고 실패 원인을 구분	자동화 실패를 빠르게 진단하고 복구

특히 MSM에서는 원문에 없는 사실을 AI가 추가하거나 사용자의 문체를 지나치게 바꾸는 문제도 확인해야 합니다. 따라서 배포 점검은 단순히 화면이 열리는지 보는 데서 끝나지 않고, 서비스가 의도한 품질로 작동하는지 확인하는 과정까지 포함해야 합니다.

8. 객관식 퀴즈 정답

정답: 2번

키 값이 리비전 설정에 그대로 기록되어 프로젝트를 볼 수 있는 사람이 콘솔에서 읽을 수 있고, 키를 바꿔도 이전 리비전에 남을 수 있습니다.

일반 환경변수로 직접 넣는 방식은 시크릿 참조 방식과 다릅니다. 키 값이 설정이나 명령 기록에 노출될 수 있으므로, 운영 환경에서는 Secret Manager를 이용해 관리하는 것이 바람직합니다.

9. 주관식 퀴즈 모범 답안

다음 답변은 제출용으로 사용할 수 있도록 간결하게 정리했습니다.

터미널에 포트 오류가 표시되더라도 포트 설정이 실제 원인이라고 단정하면 안 됩니다. 이번 사례에서는 LLM_API_KEY가 없어 앱이 시작하자마자 종료된 것이 진짜 원인이므로, 메시지만 보고 포트 설정을 수정하면 불필요한 변경이 발생하고 문제를 해결하지 못할 수 있습니다.

직접 고칠 때는 Cloud Run의 실패한 리비전 로그를 확인하고, 로그에 기록된 오류를 근거로 원인을 찾습니다. 코딩 에이전트에게 맡길 때는 로그를 근거로 원인을 설명하게 하고, 원인 확인 전에는 아무것도 수정하지 말라고 요청합니다. 해결책을 적용하기 전에는 변경 명령과 대상을 확인하고 승인합니다.

10. 핵심 요약 및 반드시 기억할 내용

- 오류 메시지와 실제 원인은 다를 수 있습니다. 터미널 메시지만 보지 말고 로그에서 원인을 찾습니다.
- 로컬 API 키는 서버에 자동으로 전달되지 않습니다. `.env`가 배포에서 제외되기 때문입니다.
- 운영용 API 키는 Secret Manager에 보관합니다. 서비스에는 시크릿 이름을 연결하고, 필요한 계정에만 읽기 권한을 줍니다.
- 실제 런타임 서비스 계정을 확인합니다. 강의 예시의 기본 계정을 무조건 사용하지 말고 서비스 설정을 확인합니다.
- 키 교체 후 새 리비전의 정상 작동을 검증합니다. `latest` 참조만으로 실행 중인 인스턴스의 키가 즉시 바뀐다고 가정하지 않습니다.
- 서비스와 컨테이너 이미지는 별도로 정리합니다. 삭제할 대상과 리전을 확인하고 필요한 리소스는 보존합니다.
- 에이전트에게는 근거 확인과 승인 절차를 요구합니다. 특히 비밀정보, 권한 변경, 삭제 작업에서는 더욱 중요합니다.

한 문장 정리

안전한 AI 서비스 운영은 오류 메시지를 그대로 믿지 않고 로그로 원인을 확인하며, API 키를 안전하게 관리하고, 중요한 변경을 검증한 뒤 적용하는 데서 시작합니다.

Chapter08

/강의정리 챕터 08. Google Cloud 배포 구조와 안전한 명령 실행

1. 오늘 강의의 목적

오늘 수업의 핵심은 AI 에이전트를 클라우드에 배포하고, 문제가 생겼을 때 원인을 찾아 안전하게 운영하는 방법을 이해하는 것입니다.

지금까지는 코드를 작성하고 실행하는 데 집중했다면, 이제는 내가 만든 AI 서비스를 다른 사람도 사용할 수 있도록 공개하고 관리하는 단계로 넘어왔습니다.

오늘 배운 내용을 네 가지로 정리할 수 있습니다.

- 클라우드 구조 이해: 계정, 결제 계정, 프로젝트, API, 리전의 관계
- 배포 과정 이해: 코드가 서비스 URL로 바뀌기까지의 단계
- 보안 이해: API 키를 로컬과 클라우드에서 안전하게 관리하는 방법
- 운영 원칙 습득: 명령을 실행하기 전에 변경 내용을 확인하고 결과를 검증하는 방법

2. 중학생도 이해할 수 있는 설명

① Google Cloud는 건물을 짓는 과정과 같습니다

Google 계정

건물을 관리하는 사람

결제 계정

건축비를 지불하는 계정

프로젝트

서비스를 만드는 작업 공간

API 및 리전 설정

필요한 시설과 건설 지역 선택

실행 중인 AI 서비스

사용자가 URL로 접속하는 서비스

여기서 중요한 점은 프로젝트 이름과 프로젝트 ID를 구분하는 것입니다. 사람이 보기 좋은 이름이 따로 있어도 명령어에서는 실제 프로젝트 ID를 사용해야 합니다.

또한 로컬 컴퓨터의 `gcloud CLI`, Google Cloud 콘솔, Cloud Shell은 서로 다른 화면과 실행 환경이지만, 같은 프로젝트를 선택하면 그 프로젝트의 리소스를 함께 관리할 수 있습니다.

② 배포 버튼 하나 뒤에는 여러 단계가 있습니다

코드를 배포한다고 해서 코드가 곧바로 인터넷에서 실행되는 것은 아닙니다.

1. 코드와 의존성 준비

프로그램과 실행에 필요한 라이브러리 확인

2. 빌드

코드를 실행할 수 있는 컨테이너 이미지로 제작

3. 이미지 저장

Artifact Registry 등에 실행 이미지 보관

4. 리비전 생성 및 인스턴스 실행

배포된 버전을 기록하고 실제 실행 환경 가동

5. 서비스 URL 제공

사용자가 접속할 수 있는 주소로 서비스 제공

이 과정을 이해하면 배포 실패 시 어디를 살펴봐야 하는지도 알 수 있습니다.

- Cloud Build 기록: 코드가 이미지로 만들어지는 과정에 문제가 있었는가?
- 리비전 탭: 어떤 버전이 만들어졌는가?
- 로그 탭: 프로그램이 실행되다가 어떤 오류를 냈는가?

배포에 성공했다는 메시지만으로 서비스가 정상 작동한다고 단정해서는 안 됩니다. 실제 실행과 기능까지 확인해야 합니다.

③ API 키는 집 열쇠처럼 관리합니다

API 키는 AI 모델을 사용하는 데 필요한 비밀 정보입니다.

로컬 컴퓨터에서는 `.env` 파일에 키를 저장할 수 있지만, 이 파일을 Git 저장소나 클라우드 배포 파일에 포함하지 않도록 설정할 수 있습니다.

- `.gitignore`: Git이 추적할 파일에서 제외
- `.gcloudignore`: 소스 배포 시 업로드할 파일에서 제외
- `Secret Manager`: 클라우드에서 비밀 정보를 안전하게 보관하고 서비스에 연결하는 기능

여기서 기억할 점은 파일을 제외하는 설정과 서버에 키를 제공하는 설정은 서로 다르다는 것입니다.

`.env` 파일을 배포하지 않았다면 서버가 그 안의 키를 자동으로 알 수 없습니다. 따라서 Secret Manager에 키를 저장하고, 서비스가 그 키를 읽을 수 있도록 연결해야 합니다.

④ 오류 메시지와 실제 원인은 다를 수 있습니다

예를 들어 배포한 프로그램이 실행되지 않아 터미널에 포트 관련 오류가 표시되었다고 가정해 보겠습니다.

실제 원인은 API 키가 없어서 프로그램이 먼저 종료된 것일 수 있습니다.

이때 포트 설정부터 바꾸면 불필요한 수정이 생길 수 있습니다.

올바른 순서는 다음과 같습니다.

1. 오류 메시지를 확인합니다.
2. Cloud Logging에서 프로그램이 실제로 종료된 이유를 찾습니다.
3. 로그에 근거해 원인을 판단합니다.
4. 수정할 대상과 변경 내용을 정합니다.
5. 명령을 실행하고 콘솔에서 결과를 확인합니다.

AI 코딩 에이전트에게 문제 해결을 맡길 때도 마찬가지입니다. 처음부터 수정하라고 하기보다, 먼저 로그를 근거로 원인을 설명하고 아직 수정하지 말라고 지시하는 것이 안전합니다.

3. 비유로 이해하기: 배포는 식당 개업과 같습니다

클라우드 개념	식당에 비유하면
프로젝트	식당 사업장
API 활성화	필요한 시설 사용 허가
컨테이너 이미지	조리법과 장비를 갖춘 표준 주방 세트
리비전	메뉴와 운영 방식이 기록된 버전
인스턴스	실제 영업 중인 주방
서비스 URL	손님이 찾아오는 주소
Secret Manager	금고에 보관한 열쇠와 비밀번호
로그	영업 중 발생한 문제를 기록한 일지
롤백	문제가 생기기 전의 운영 방식으로 복구

식당에서 새 메뉴를 시험하다 문제가 생기면 이전 메뉴로 돌아갈 수 있습니다. 클라우드에서도 이전 리비전으로 트래픽을 돌려보내 비슷한 효과를 얻을 수 있습니다.

4. 실제 활용 예시: 배포 후 문제가 생겼을 때

가령 MSM(My-Story-Maker)의 글 분석 기능을 배포했는데 실행되지 않는다고 가정하겠습니다.

1. 증상 확인: 서비스 URL에 접속했지만 글 분석 기능이 작동하지 않습니다.

2. 로그 확인: 프로그램이 API 키를 찾지 못했다는 오류를 확인합니다.
3. 원인 구분: 포트 오류처럼 보이더라도 실제로는 키 누락일 수 있습니다.
4. 안전하게 수정: Secret Manager의 키와 서비스 연결, 권한을 확인합니다. 비밀 값 자체는 대화창이나 코드에 붙이지 않습니다.
5. 재배포 및 검증: 새 리비전이 생성되었는지, 정상 실행되는지, 실제 글 분석이 되는지 확인합니다.

이 과정은 MSM뿐 아니라 ASP의 AI 상담 서비스와 뉴스레터 자동화 에이전트에도 그대로 적용할 수 있습니다.

5. AI 개발자가 반드시 알아야 하는 이유

AI 개발자는 코드를 작성하는 능력만으로 충분하지 않습니다. 실제 사용자가 서비스를 이용할 수 있도록 배포하고, 장애를 진단하고, 비밀 정보를 보호할 수 있어야 합니다.

특히 AI 에이전트가 명령을 대신 실행하는 환경에서는 다음 세 가지가 중요합니다.

- 변경 범위: 어떤 프로젝트와 서비스에 영향을 주는가?
- 변경 결과: 무엇을 생성·수정·삭제하거나 외부에 공개하는가?
- 검증 방법: 실행 후 정상 작동을 어떻게 확인할 것인가?

명령을 직접 입력하든 AI 에이전트에게 맡기든, 이 원칙은 같습니다. AI가 명령을 작성할 수 있어도 실행 결과에 대한 책임까지 자동으로 해결해 주는 것은 아닙니다.

6. 나의 학습·창업 목표와 연결하기

사용자님의 목표인 1인 AI 사업과 실제 프로젝트를 기준으로 연결해 보겠습니다.

프로젝트	오늘 배운 내용의 적용
MSM	글 분석 및 변환 기능을 클라우드에 배포하고, API 키와 실행 오류를 안전하게 관리
ASP	고객이 웹사이트에서 AI 상담을 이용할 수 있도록 배포하고 서비스 상태 확인
뉴스레터 에이전트	자동 실행 중 발생하는 오류를 로그로 확인하고, 변경 전후의 동작을 검증
AI 컨설팅	고객사에 배포 구조, API 키 관리, 장애 대응 및 운영 안전성까지 설명

특히 사용자님은 28년간 신사업 기획과 프로젝트 관리 경험이 있으십니다. 이 경험은 클라우드 운영에서도 강점이 됩니다. 개발자가 모든 명령을 기억하는 것보다, 무엇을 바꾸려는지 정의하고 위험을 확인하며 결과를 검증하는 능력이 중요하기 때문입니다.

앞으로 MSM을 만들 때도 다음 질문을 습관화해 보시면 좋겠습니다.

- 이 기능은 어떤 프로젝트와 서비스에 배포되는가?
- API 키는 어디에 저장되고, 어떤 계정이 읽을 수 있는가?
- 배포가 실패하면 어디서 원인을 확인하는가?
- 문제가 생기면 이전 버전으로 되돌릴 수 있는가?

7. 주관식 퀴즈 모범 답안

Q. Vercel 배포 버튼을 누른 뒤 일어나는 일을 처음 짐작했던 내용과 오늘 Google Cloud 콘솔에서 직접 확인한 내용을 비교해 보세요.

아래는 제출용으로 활용할 수 있는 예시 답안입니다. 실제로 Vercel 배포를 처음 추측했던 답안은 현재 제공되지 않았으므로, 본인이 당시 무엇을 예상했는지에 맞게 수정하시면 됩니다.

> 처음에는 배포 버튼을 누르면 작성한 코드가 서버에 올라가고 곧바로 웹사이트가 공개되는 것으로 생각했습니다. 오늘 Google Cloud 콘솔에서 직접 확인해 보니, 실제로는 코드를 빌드하고 컨테이너 이미지를 저장한 뒤 리비전을 만들고 인스턴스를 실행하여 서비스 URL을 제공하는 여러 단계가 있었습니다.
>
> 코드가 서버에 올라가고 웹사이트가 공개된다는 큰 흐름은 맞았습니다. 하지만 배포 과정에서 여러 리소스가 만들어지고, 각 단계의 상태와 오류를 Cloud Build 기록·리비전·로그에서 따로 확인할 수 있다는 점은 새롭게 알게 되었습니다. 또한 API 키가 배포 파일에 포함되지 않았다면 서버에서 별도로 안전하게 연결해야 하며, 배포 성공 여부뿐 아니라 실제 서비스가 정상적으로 작동하는지도 검증해야 한다는 점을 배웠습니다.

8. 핵심 요약

클라우드 구조

계정 → 결제 계정 → 프로젝트 → API·리전 → 서비스의 관계를 이해합니다.

배포 과정

빌드 → 이미지 저장 → 리비전 → 인스턴스 실행 → 서비스 URL 순서로 이해합니다.

보안과 장애 대응

API 키는 Secret Manager로 관리하고, 오류는 로그를 근거로 진단합니다.

안전한 실행

명령의 대상과 영향을 먼저 확인하고, 실행 후 콘솔에서 결과를 검증합니다.

9. 반드시 기억할 내용

1. 프로젝트 이름이 아니라 프로젝트 ID를 사용하는 명령이 많습니다.
2. 배포 명령 하나 뒤에서도 빌드, 이미지 저장, 리비전 생성 등 여러 단계가 진행됩니다.
3. `.gitignore`와 `.gcloudignore`는 키를 서버에 전달하는 기능이 아닙니다.
4. 터미널에 보이는 오류 문구가 실제 원인과 다를 수 있으므로 로그를 확인해야 합니다.
5. AI 에이전트가 명령을 실행하기 전에는 변경 대상과 영향을 확인해야 합니다.
6. 배포 성공 메시지와 실제 서비스 정상 작동은 별개입니다.

10. 한 문장 정리

클라우드 배포는 코드를 공개하는 버튼 하나의 문제가 아니라, 실행 환경과 비밀 정보를 안전하게 준비하고, 버전을 관리하며, 로그를 근거로 문제를 해결하고 결과를 검증하는 운영 과정입니다.